A DUAL-MICROSERVICE PRICING ENGINE FOR TRANSPARENT TRANSPORT FEES AND REPAIR COST ESTIMATION IN A SRI LANKAN HOUSEHOLD SERVICE MARKETPLACE

INDIVIDUAL THESIS

Kumara A.J.T.M

(IT22137982)

Individual component: Dynamic Transport Fee and Repair Price Estimation Engine

Group ID:R26-IT-090

BSc (Hons) in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Kumara A.J.T.M

Student ID: IT22137982

Signature:  Date: 2026/10/28

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Dasuni Navinna		2026/10/28
Co-supervisor: Mr. Nelum Chathuranga Amarasena		2026/10/28

ABSTRACT

Household maintenance work in Sri Lanka is largely informal, and two pricing problems weaken on-demand service platforms. Technicians are reluctant to travel because traffic delays, rain and fuel price increases are not compensated, and customers cannot judge whether a repair quotation is fair. This thesis presents Component 4 of an AI-powered household service marketplace: a pricing engine built as two independent Python microservices. The Context-Aware Dynamic Transport Fee Engine (Flask) calculates an itemised cost-plus fee from a base fare of LKR 200, the route distance from the Google Maps Distance Matrix API, a rate of LKR 80 per kilometre indexed to the CEYPETCO petrol price, a three-level traffic multiplier (1.00, 1.15, 1.30) and a weather multiplier from OpenWeatherMap (1.00, 1.20, 1.35). The Repair Price Range Estimator (FastAPI) predicts a minimum and maximum price for a worker type, service and specification with a one-hot encoded, multi-output decision tree. It was trained on 630 rows derived from 105 surveyed Sri Lankan price combinations covering 39 trades, 95 services and 91 specifications.

On a random 80/20 hold-out split, the decision tree reached a coefficient of determination (R^2) of 0.888 for the minimum price (mean absolute error LKR 1,712) and 0.925 for the maximum price (LKR 3,144). A 100-tree random forest gave similar R^2 values (0.890 and 0.916) but higher mean absolute error on both outputs. Augmented copies of the same base combination could fall in both the training and test sets, so these figures are treated as optimistic. Both services run on the Render cloud platform and pass 14 automated tests, and the estimator's mean inference latency was 11.10 ms. A scenario analysis shows that the transport fee responds predictably and explainably to distance, congestion, weather and the 2026 fuel price revisions. Validation with real customers and technicians remains future work.

Keywords: dynamic pricing, repair price estimation, decision tree regression, microservices, household services

ACKNOWLEDGEMENTS

I am sincerely grateful to my supervisor, Prof. Dasuni Navinna, and my co-supervisor, Mr. Nelum Chathuranga Amarasena, for their guidance, encouragement and constructive feedback throughout this research. Their questions about fairness, data quality and evaluation shaped both the pricing engine and the way its limitations are reported in this thesis.

I thank the members of research group R26-IT-090, whose intent recognition, worker recommendation and trust scoring components made it possible to place this pricing engine inside a complete service marketplace. I am also thankful to the independent tradespeople and repair workshops who shared their typical labour charges and job quotations; their information is the foundation of the repair price dataset.

Finally, I thank the academic and administrative staff of the Faculty of Computing, Sri Lanka Institute of Information Technology, and my family and friends for their constant support. This research received no external funding.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i
ABSTRACT ii
ACKNOWLEDGEMENTS iii
TABLE OF CONTENTS iv
LIST OF FIGURES viii
LIST OF TABLES ix
LIST OF ABBREVIATIONS xi
1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 2
1.3 Problem statement 3
1.3.1 Uncompensated travel friction for technicians 3
1.3.2 Price information asymmetry for customers 4
1.4 Research gap 4
1.5 Aim and objectives 5
1.6 Research questions or hypotheses 5
1.7 Scope limitations and assumptions 6
1.8 Individual contribution 7
1.9 Thesis structure 7
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review method and scope 9
2.2 Domain and theoretical foundations 9
2.2.1 Information asymmetry and credence goods 9
2.2.2 Price fairness and transparency 10
2.2.3 Pricing strategies: cost-plus, value-based and dynamic pricing 11
2.2.4 Travel time, congestion and weather 11
2.2.5 Decision trees, ensembles and multi-output regression 12
2.2.6 Prediction intervals and price ranges 12
2.3 Related research 13
2.3.1 Dynamic and algorithmic pricing on service platforms 13
2.3.2 Cost-based fare structures in Sri Lanka 13
2.3.3 Machine learning for cost and price estimation 14
2.3.4 Evaluation practice and data leakage 15

2.4 Existing systems products and techniques 15
2.5 Gap synthesis and conceptual framework 16
3 RESEARCH METHODOLOGY 18
3.1 Research design 18
3.2 Data participants or study materials 19
3.2.1 Repair price dataset 19
3.2.2 Data augmentation 20
3.2.3 Data for the transport fee engine 21
3.2.4 Participants 21
3.3 Experimental or study procedure 22
3.3.1 Repair price model experiment 22
3.3.2 Transport fee scenario analysis 23
3.3.3 Software verification and latency measurement 23
3.4 Measures and evaluation metrics 24
3.5 Data analysis 25
3.5.1 Data leakage in the evaluation design 25
3.5.2 Analysis of the transport fee 25
3.6 Ethics privacy security and responsible AI 25
3.7 Reproducibility and research artefacts 26
4 PRODUCT AND SOLUTION DESIGN 28
4.1 Requirements 28
4.2 Component architecture within the complete solution 29
4.3 Design alternatives and decisions 30
4.4 Detailed design 31
4.4.1 Transport fee model 31
4.4.2 Transport fee processing flow 33
4.4.3 Transport fee API contract 34
4.4.4 Repair price model 35
4.4.5 Repair estimate API contract 36
4.4.6 Interaction design 36
4.4.7 Error handling design 37
4.5 Standards and professional practice 38
5 IMPLEMENTATION INTEGRATION AND TESTING 39
5.1 Development environment and technology stack 39
5.2 Component implementation 39

5.2.1 Transport fee engine 39
5.2.2 Repair price model training 40
5.2.3 Repair price estimator service 41
5.3 Integration with the other components 41
5.4 Verification and software testing 42
5.5 Deployment and operation 43
6 RESULTS AND VALIDATION 45
6.1 Research results 45
6.1.1 Accuracy of the repair price models 45
6.1.2 Validity of the predicted ranges 47
6.2 Product and technical results 47
6.2.1 Behaviour of the transport fee engine 47
6.2.2 Sensitivity to fuel prices 49
6.2.3 Performance and reliability 50
6.3 User or stakeholder validation 51
6.4 Integration results 51
6.5 Summary of findings 52
7 DISCUSSION 53
7.1 Interpretation of findings 53
7.1.1 Transport fee engine 53
7.1.2 Repair price model 53
7.2 Comparison with literature baselines and existing solutions 55
7.3 Research contribution 56
7.4 Product contribution 56
7.5 Limitations and threats to validity 57
7.6 Implications 58
8 COMMERCIALISATION AND SUSTAINABILITY 59
8.1 Value proposition and target users 59
8.2 Market and competitive position 59
8.3 Business or sustainability model 60
8.4 Scalability intellectual property and compliance 61
8.5 Contribution to the Sustainable Development Goals 62
9 CONCLUSION AND FUTURE DIRECTIONS 64
9.1 Conclusion 64
9.2 Achievement of objectives 65

9.3 Individual research and product contributions 65
9.4 Future directions 65
REFERENCES 67
GLOSSARY 73
APPENDICES 74
Appendix A Extended results 74
Appendix B Academic integrity and AI use disclosure 75

LIST OF FIGURES

Figure 1.1 Position of Component 4 within the complete marketplace solution 2
Figure 2.1 Conceptual framework of Component 4 17
Figure 3.1 Design science research process followed in this study (adapted from Peffers et al. [64]) 18
Figure 3.2 Dataset construction and evaluation pipeline for the repair price model 23
Figure 4.1 Internal architecture and deployment of Component 4 30
Figure 4.2 Step function mapping the traffic ratio to the traffic multiplier 32
Figure 4.3 Processing flow of the transport fee engine 34
Figure 4.4 Repair price estimation pipeline 35
Figure 4.5 Sequence of pricing interactions during a booking 37
Figure 6.1 MAE and RMSE of the decision tree and random forest for each target 45
Figure 6.2 R^2 of the decision tree and random forest for each target (axis starts at 0.85) 46
Figure 6.3 Transport fee against route distance for four travel conditions, with the three-wheeler tariff as a reference 48
Figure 6.4 Composition of the LKR 1,109 example fee (5.4 km, heavy traffic, rain) 49
Figure 6.5 Fee for a 10 km trip in normal conditions under the petrol prices of 2026 50
Figure 6.6 Measured inference latency of the repair price estimator 50

LIST OF TABLES

Table 1.1 Specific objectives of Component 4 5
Table 2.1 Summary of related research..... 14
Table 2.2 Pricing approaches of existing systems and practices..... 15
Table 3.1 Structure of the repair price dataset 20
Table 3.2 Representative trade, service and specification combinations in the dataset 20
Table 3.3 Augmentation rules applied to each base combination..... 21
Table 3.4 Evaluation measures used in this study..... 24
Table 3.5 Research artefacts produced in this study..... 26
Table 4.1 Functional requirements..... 28
Table 4.2 Non-functional requirements 28
Table 4.3 Design alternatives and decisions 30
Table 4.4 Parameters of the transport fee model..... 31
Table 4.5 Traffic multiplier rules..... 32
Table 4.6 Weather multiplier rules..... 33
Table 4.7 Error handling and fallback behaviour..... 37
Table 5.1 Technology stack of Component 4 39
Table 5.2 Automated test coverage and results..... 43
Table 6.1 Hold-out results of the repair price models (random 80/20 split of the augmented dataset)..... 45
Table 6.2 Transport fee (LKR) by distance and travel conditions..... 47
Table 6.3 Fee for a 10 km trip in normal conditions under 2026 petrol prices..... 49
Table 6.4 Planned user and stakeholder validation..... 51
Table 6.5 Integration status of Component 4..... 51
Table 6.6 Summary of findings by research question..... 52
Table 7.1 Limitations and threats to validity..... 57
Table 8.1 Value proposition by user group..... 59
Table 8.2 Competitive position of the pricing approach..... 60
Table 8.3 Business model canvas for the pricing engine..... 60
Table 8.4 Contribution to the Sustainable Development Goals..... 62
Table 9.1 Achievement of the specific objectives..... 65
Table A.1 Transport fee (LKR) by distance and multiplier combination 74

Table A.2 Mapping of OpenWeatherMap condition groups to the weather multiplier 74
Table A.3 Worked transport fee calculations..... 75
Table B.1 Disclosure of AI and automated tool use 75

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BTU	British Thermal Unit
CART	Classification and Regression Trees
CEYPETCO / CPC	Ceylon Petroleum Corporation
CORS	Cross-Origin Resource Sharing
DSR	Design Science Research
DT	Decision Tree
FR	Functional Requirement
GPS	Global Positioning System
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LKR	Sri Lankan Rupee
MAE	Mean Absolute Error
ML	Machine Learning
NFR	Non-Functional Requirement
PaaS	Platform as a Service
PDPA	Personal Data Protection Act (Sri Lanka)
REST	Representational State Transfer
RF	Random Forest
RMSE	Root Mean Square Error
RPTA	Road Passenger Transport Authority

Abbreviation	Meaning
RQ	Research Question
R^2	Coefficient of determination
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
TTI	Travel Time Index
WP	Western Province
WSGI	Web Server Gateway Interface

1 INTRODUCTION

1.1 Overall project context

Household maintenance in Sri Lanka, including plumbing, electrical work, air-conditioner servicing, carpentry, masonry and appliance repair, is delivered mainly by independent tradespeople and small workshops. Customers usually find these workers through relatives, neighbours, roadside signboards or classified advertisement sites, and agree a price informally. The services sector employed 50.6% of the employed population in the fourth quarter of 2025 [1], yet very little of the household repair segment is organised through digital platforms. This leaves customers without reliable information about who to call and what to pay, and leaves capable workers without a steady flow of jobs.

The group research project responds to this situation with an AI-powered smart service marketplace for household maintenance in Sri Lanka. The solution is a Flutter mobile application for customers and technicians, supported by a Node.js backend with a MongoDB Atlas database and four intelligent Python services. Component 1 recognises the customer's intent from free-text English or romanised Sinhala (Singlish) problem descriptions and maps it to one of 39 worker categories. Component 2 recommends suitable workers using a hybrid of a Random Forest competence model and a real-time model of distance and ratings. Component 3 builds a review credibility and sentiment-based trust score from verified reviews. Component 4, the subject of this thesis, deals with the money side of a booking: what the technician should be paid for travelling to the customer, and what the customer should expect to pay for the repair itself. Figure 1.1 shows how the four components fit into the complete solution.

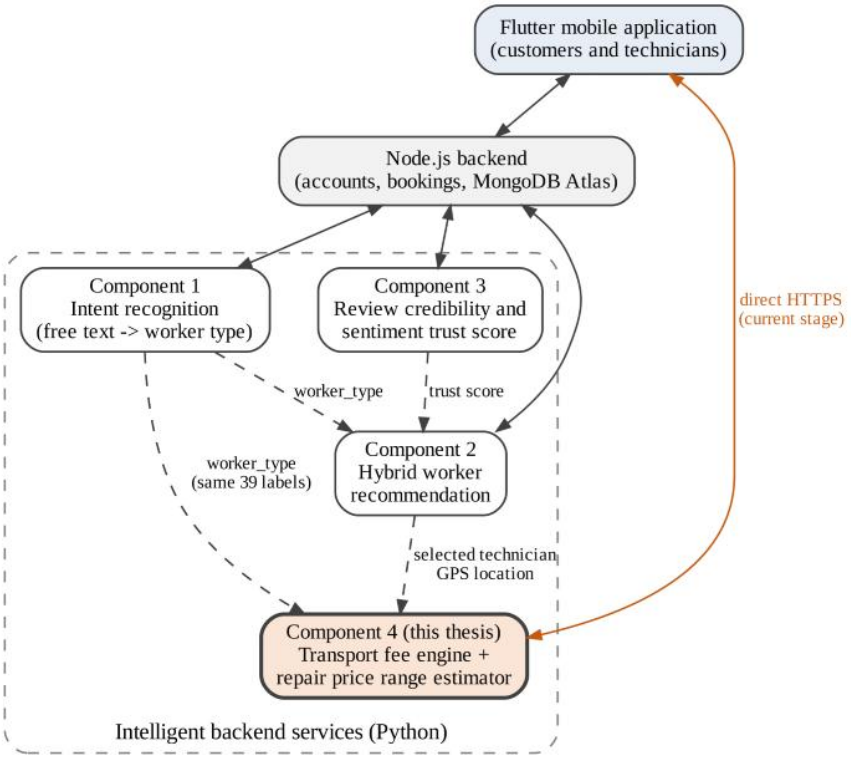


Figure 1.1: Position of Component 4 within the complete marketplace solution

1.2 Individual component context

Pricing is the point at which a marketplace either earns or loses the trust of both sides. Ride-hailing platforms showed that a transparent, algorithmic fare can replace street bargaining for transport [2], [3], but household repair has two separate price elements. The first is the cost of bringing a skilled worker to the house, which depends on distance, traffic, weather and fuel. The second is the cost of the repair work, which depends on the trade, the task and its size. Component 4 therefore contains two decoupled microservices:

- **Module 1 – Context-Aware Dynamic Transport Fee Engine.** A synchronous Flask service that calculates an itemised travel surcharge from the technician's location to the customer's home. It uses live route distance and traffic duration from the Google Maps Distance Matrix API, live weather at the destination from OpenWeatherMap, and the current Ceylon Petroleum Corporation (CEYPETCO) petrol price.
- **Module 2 – Machine Learning Repair Price Range Estimator.** An asynchronous FastAPI service that predicts a minimum and maximum repair

price in Sri Lankan rupees (LKR) for a combination of worker type, service and specification, using a multi-output decision tree trained on surveyed local market rates.

Both services are deployed independently on the Render cloud platform and are called by the Flutter application through REST endpoints. The repair estimate is shown when the customer selects a service, before any technician has been chosen. The transport fee is calculated at checkout, after the customer selects a technician from Component 2's recommendations, using that technician's current GPS location as the origin and the customer's home as the destination. The worker category used by Module 2 is the same set of 39 labels produced by Component 1, so the two components share a common vocabulary.

1.3 Problem statement

Component 4 addresses two related problems that appear at the moment of booking.

1.3.1 Uncompensated travel friction for technicians

A technician who accepts a job must travel to the customer, usually by motorcycle or three-wheeler, carrying tools and parts. The cost of that journey is not fixed. It grows with distance, and it grows further when roads are congested or when it rains, because travel takes longer and is more dangerous [4], [5]. Fuel costs have been a particular concern since the 2022 economic crisis, when fuel shortages and very high inflation sharply raised the cost of transport [6]. Fuel prices have also changed frequently since then: the retail price of Petrol 92 Octane was revised from LKR 398 to LKR 410 per litre on 2 May 2026 [7], to LKR 434 on 30 May 2026 [8] and from LKR 399 to LKR 414 on 1 October 2026 [9]. When a platform pays a flat call-out charge, or nothing at all for travel, technicians rationally avoid distant jobs, jobs during rush hour and jobs in bad weather. Evidence from ride-hailing shows that supply responds strongly to price in exactly these conditions [10], [11]. A fixed travel charge therefore leads to unfilled requests at the times customers need help most.

1.3.2 Price information asymmetry for customers

Most customers cannot judge what a repair should cost. Household repair has the features of a credence good: the expert diagnoses the problem and also sells the fix, and the customer often cannot verify whether the work or the price was appropriate [12], [13]. Studies of automobile repair have found that this information gap leads to over-treatment and over-charging, which reputation alone does not fully correct [14]. In Sri Lanka the gap is wider because repair prices are rarely published, quotations are verbal, and labour rates vary between districts and between workers. A customer who has no reference price either accepts the first quotation or spends time bargaining, and both outcomes reduce trust in the platform. Behavioural research also shows that people judge prices against a reference point and react strongly to prices they see as unfair [15], [16].

Taken together, the problem addressed in this thesis is: **the marketplace lacks a transparent, context-aware and locally grounded mechanism for pricing both the technician's travel and the repair work, which discourages technicians from accepting jobs and prevents customers from forming fair price expectations before the technician arrives.**

1.4 Research gap

Chapter 2 reviews the literature in detail; the gap is summarised here. Dynamic pricing research has concentrated on ride-hailing, where the price is set to balance passenger demand and driver supply in real time [2], [17], [18]. These surge models are not designed for household services, where the main service is performed at the destination and travel is a secondary cost, and they are often criticised as opaque [3], [19]. In Sri Lanka, the only published work on cost-based transport fares that was found concerns three-wheeler passenger fares and does not consider traffic, weather or fuel indexation in real time [20].

On the repair side, machine learning has been applied to construction cost estimation [21], residential building repair costs [22], housing prices [23] and used car prices [24]. These studies predict a single value from rich numerical features, mostly with data from developed countries. No study was found that predicts a **price range** (minimum and maximum) for **household repair services** from purely categorical job

descriptors, with data collected from **Sri Lankan** tradespeople, and that is deployed as a service inside a marketplace application. The research gap is therefore the combination of (i) a transparent, context-aware travel fee for household technicians, and (ii) a locally grounded, interpretable repair price range estimator, both delivered as low-latency microservices.

1.5 Aim and objectives

Aim. To design, implement and evaluate a dual-microservice pricing engine that gives technicians a fair, context-aware transport fee and gives customers a data-driven repair price range, so that both parties can agree a booking with transparent price expectations.

The specific objectives are listed in Table 1.1.

Table 1.1: Specific objectives of Component 4

ID	Objective	Measure of achievement
O1	Review literature on dynamic pricing, information asymmetry, travel-time effects and tree-based price estimation to justify the design.	Literature synthesis and conceptual framework (Chapter 2)
O2	Design a deterministic cost-plus transport fee model that uses route distance, fuel price indexation, live traffic and weather.	Formula, parameter values and multiplier rules defined (Chapter 4)
O3	Build a repair price dataset for 39 worker types from Sri Lankan market rates and train a multi-output model to predict minimum and maximum price.	630-row dataset; trained decision tree pipeline (Chapters 3 and 5)
O4	Compare the decision tree with a random forest baseline using MAE, RMSE and R^2.	Hold-out results for both models (Chapter 6)
O5	Deploy both modules as independent cloud microservices with clear REST contracts, fault tolerance and server-side secret management.	Live Render endpoints; 14 automated tests passed; latency measured
O6	Integrate the services with the mobile application and the other components, and assess commercial viability.	Integration flow (Chapter 5); commercialisation analysis (Chapter 8)

1.6 Research questions or hypotheses

The work is guided by three research questions (RQ):

- **RQ1.** Can a deterministic, context-aware cost-plus model that combines route distance, fuel price indexation, live traffic and weather produce a transparent, itemised transport fee suitable for real-time dispatch in Sri Lanka?

- **RQ2.** How accurately can a multi-output decision tree trained on surveyed Sri Lankan market rates estimate the minimum and maximum price of a household repair job from its worker type, service and specification, and how does it compare with a random forest?
- **RQ3.** Can both models be delivered as independent cloud microservices that respond fast enough for an interactive mobile application and remain usable when external services fail?

For RQ2, the working hypothesis was that a single decision tree would match the accuracy of a random forest on this categorical problem while being simpler to explain to users. The results in Chapter 6 partly support this hypothesis: the two models were comparable, with the decision tree having lower MAE and the random forest a slightly higher R^2 for the minimum price.

1.7 Scope limitations and assumptions

The scope of this thesis is limited as follows:

- The transport fee is calculated for one-way travel from the technician's current location to the customer. Return travel, waiting time, parking and tolls are not priced separately.
- Fee parameters are configured values: base fare LKR 200, distance rate LKR 80 per km, baseline petrol price LKR 350 per litre and current petrol price LKR 414 per litre (CEYPETCO Petrol 92 Octane from 1 October 2026 [9]). The current price is updated manually in the server environment.
- Weather is taken at the destination only, and only rain or drizzle and thunderstorms change the fee.
- The repair price estimator covers the 39 worker types, 95 services and 91 specifications present in the dataset. It predicts labour and typical job cost ranges; it does not diagnose faults or price specific spare parts.
- The repair dataset was derived from 105 surveyed base combinations expanded to 630 rows by synthetic augmentation. The model is evaluated on a random split of the augmented data, which is known to give optimistic results (Section 3.5).

- The services were tested with automated tests and by the development team. No study with real customers or technicians has been carried out yet.
- The final booking price, payment processing and platform commission are handled by the wider platform and are outside the scope of this component.

The main assumptions are that Google Maps traffic durations are a reasonable proxy for real travel effort, that technicians travel mainly by motorcycle or three-wheeler so that fuel cost scales with petrol prices, and that the surveyed price ranges represent typical urban market rates in Sri Lanka in 2026.

1.8 Individual contribution

This component was designed, implemented, deployed and evaluated by the author. The individual contributions are:

- Design of the context-aware cost-plus transport fee model, including the choice of parameters, the traffic thresholds (15% and 40% delay) and the weather multipliers, together with the itemised response format.
- Implementation of the Flask transport service with Google Maps and OpenWeatherMap integration, coordinate validation, timeout fallback and environment-based configuration.
- Collection of repair price information from tradespeople and workshops, cross-checking against published service listings, and construction of the 630-row repair price dataset aligned with Component 1's 39 worker categories.
- Training and comparison of the multi-output decision tree and random forest models, and implementation of the FastAPI estimator service.
- Containerisation and deployment of both services on Render, 14 automated tests with pytest and Postman, and latency measurement.
- Integration of the two endpoints into the Flutter booking flow and alignment with Components 1 and 2.

1.9 Thesis structure

Chapter 2 reviews the theory and related work on pricing, travel conditions and tree-based price estimation, and identifies the research gap. Chapter 3 describes the

research methodology, the dataset and the evaluation procedure, including the data leakage risk. Chapter 4 presents requirements, architecture and the detailed design of both modules. Chapter 5 covers implementation, integration, testing and deployment. Chapter 6 reports the results, and Chapter 7 discusses them against the literature and states the limitations. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes with achievements and future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review followed a structured but non-systematic approach. Searches were made in Google Scholar, IEEE Xplore, the ACM Digital Library, ScienceDirect and the INFORMS journals, using combinations of the terms *dynamic pricing*, *surge pricing*, *cost-plus pricing*, *credence goods*, *price fairness*, *travel time weather*, *traffic congestion index*, *repair cost estimation*, *price prediction machine learning*, *decision tree regression*, *multi-output regression* and *data leakage*. Backward citation searching was used from key papers such as [2] and [13]. Because the component works with Sri Lankan prices and regulations, Sri Lankan sources were also reviewed: national labour statistics [1], published fuel price revisions [7]–[9], the Western Province three-wheeler tariff [25] and local academic work on fare structures [20]. Technical documentation for the external services and frameworks was used for design details [26]–[29].

Sources were included if they (i) explained how prices are formed or perceived in service markets, (ii) measured how traffic or weather affect travel and transport supply, (iii) applied machine learning to price or cost estimation, or (iv) addressed evaluation practice for such models. Peer-reviewed work was preferred; working papers and professional articles were used only where they report widely cited industry evidence. The review is organised into theoretical foundations (Section 2.2), related research (Section 2.3), existing systems (Section 2.4) and a gap synthesis (Section 2.5).

2.2 Domain and theoretical foundations

2.2.1 Information asymmetry and credence goods

Akerlof's analysis of the market for "lemons" showed that when sellers know more about quality than buyers, prices stop carrying reliable information and trust in the market falls [30]. Darby and Karni extended this to *credence goods*, where the buyer cannot judge the quality or necessity of a service even after it is delivered, and showed that this creates room for fraud even in competitive markets [12]. Dulleck and Kerschbamer reviewed the economics of credence goods and identified repair

services, including mechanics and computer specialists, as typical cases. They described three forms of expert misbehaviour: over-treatment, under-treatment and over-charging [13]. Schneider's field study of auto repair found that repair shops commonly recommended unnecessary work and missed needed work, and that reputation did not remove these problems [14].

Household repair in Sri Lanka has the same structure. The technician diagnoses the fault and then sells the repair, and the customer seldom knows the market price. A published reference price does not remove the asymmetry, because the technician still decides what work is needed, but it gives the customer an anchor against which a quotation can be judged. Tversky and Kahneman showed that numerical anchors strongly influence later judgements [31]. A credible price range shown before the visit can therefore set expectations and reduce the chance that a fair quotation is rejected or an unfair one is accepted.

2.2.2 Price fairness and transparency

Kahneman, Knetsch and Thaler found that people accept price increases that are justified by higher costs, but see increases that exploit higher demand as unfair [15]. Xia, Monroe and Cox built on this to develop a framework of price fairness perceptions. In their framework, perceived fairness depends on comparison with reference prices and on whether the seller's reasons are visible and legitimate [16]. Soh, Markus and Goh studied electronic marketplaces and found that the level of price transparency a market offers is a strategic choice with large effects on participation [32].

These findings matter for the transport fee. A fee that rises because the road is congested or because it is raining reflects a real increase in the technician's cost and risk. It is more likely to be accepted than a fee that rises simply because many customers are requesting help at the same time. This is the main reason Component 4 uses a cost-based design with an itemised breakdown, rather than a demand-based surge multiplier.

2.2.3 Pricing strategies: cost-plus, value-based and dynamic pricing

Pricing strategies are commonly grouped into cost-based, competition-based and customer value-based approaches. Hinterhuber argued that value-based pricing can capture more value, but that many firms prefer cost-based methods because costs are easier to measure and justify [33]. Cost-plus pricing sets the price as a cost estimate plus a margin, which makes it simple to explain and audit.

Dynamic pricing changes prices over time in response to demand, supply or context. Den Boer's review traces its origins in revenue management and describes how modern systems learn demand functions from data [34]. In two-sided platforms, Cachon, Daniels and Lobel modelled a service platform whose workers choose when to work. They showed that surge pricing can raise the welfare of customers, workers and the platform by drawing more workers in at busy times [2]. Castillo, Knoepfle and Weyl showed that without surge pricing, drivers can be sent on long trips to collect passengers far away, a "wild goose chase" that wastes capacity [17]. The transport fee in this thesis combines both ideas: it is cost-plus in structure, but the cost factors are updated in real time from the context of each trip.

2.2.4 Travel time, congestion and weather

Congestion is often measured by the Travel Time Index (TTI), the ratio of travel time in peak conditions to free-flow travel time. The Texas A&M Urban Mobility Report uses this index to compare congestion between cities [35]. The traffic multiplier in Component 4 uses the same idea: the ratio of Google Maps' *duration in traffic* to its normal duration [26].

Weather also affects travel. Tsapakis, Cheng and Bolbol measured urban travel times in London and found that rain increased them, by 0.1–2.1% for light rain and 1.5–3.8% for moderate rain, with larger effects for snow [4]. Koetse and Rietveld reviewed empirical studies and concluded that rain reduces speeds and raises accident risk [5]. For a technician on a motorcycle or three-wheeler, rain also means discomfort, slower riding and risk to tools and parts. Brodeur and Nield found that in New York the number of Uber rides per hour was roughly a fifth to a quarter higher when it rained, and attributed this to surge pricing drawing more drivers onto the road; taxi rides, with fixed fares, rose only slightly [11]. A weather premium is

therefore justified both as compensation for higher cost and risk and as an incentive to keep supply available in bad weather.

2.2.5 Decision trees, ensembles and multi-output regression

Classification and Regression Trees (CART) split the feature space recursively into regions, and predict a constant value in each region; for regression the constant is the mean of the training targets in the leaf [36]. Quinlan's ID3 established the use of information-based splitting criteria [37]. Trees are easy to interpret because each prediction can be traced to a short sequence of rules. They are, however, unstable: small changes in the data can change the tree a great deal [38]. Random forests reduce this variance by averaging many trees grown on bootstrap samples with random feature selection [39]. The averaging improves accuracy in many tasks, but it also blends predictions from different regions, so a forest's output is harder to explain.

Multi-output regression predicts several continuous targets at once. Borchani et al. reviewed the methods and noted that tree-based models can handle several outputs in one model by using a split criterion averaged over the outputs, which keeps related targets consistent [40]. Scikit-learn's decision tree and random forest regressors support multi-output targets directly [41]. For a price range this matters: predicting the minimum and maximum together from the same leaf keeps the two bounds linked to the same group of similar jobs.

Rudin argued that for high-stakes decisions, interpretable models should be preferred to black-box models whenever their accuracy is comparable [42]. Doshi-Velez and Kim discussed how interpretability supports trust and accountability [43]. Grinsztajn, Oyallon and Varoquaux showed that tree-based models remain strong on typical tabular data compared with deep learning [44]. Categorical variables such as worker type and service are usually one-hot encoded for these models. Cerda and Varoquaux discuss the problems this causes when categories are many and sparse [45].

2.2.6 Prediction intervals and price ranges

A price range can be produced in two main ways. The first is to predict statistical quantiles of the price distribution, for example by quantile regression [46] or quantile

regression forests [47]. This needs many observed prices for each type of job. The second is to treat the lower and upper bounds as two targets and learn them directly, which is possible when the training data already records a typical minimum and maximum for each job. Component 4 uses the second approach because the surveyed data was collected as minimum and maximum quotations rather than as individual transactions.

2.3 Related research

2.3.1 Dynamic and algorithmic pricing on service platforms

Chen and Sheldon studied Uber driver behaviour and found that drivers worked more during surge periods, showing that labour supply on platforms responds to price signals [10]. Hall, Kendrick and Nosko analysed a New Year's Eve in New York when Uber's surge pricing failed for about 26 minutes. During the outage, the share of completed requests fell sharply and estimated waiting times rose, which they cited as evidence that dynamic prices keep the market working [18]. Rosenblat and Stark interviewed drivers and analysed forums, and found that the opacity of algorithmic pricing created information asymmetries that disadvantaged workers [3]. Dholakia described customer anger at surge pricing and recommended clearer explanation and limits on multipliers [19]. Wood et al. found that algorithmic control in the gig economy can give workers flexibility but also produces overwork and insecurity [48]. Hannák et al. found evidence of bias in worker reviews and rankings on TaskRabbit and Fiverr, showing that platform algorithms can affect fairness between workers [49].

Two lessons follow. Price signals do affect whether workers accept jobs, so a well-designed travel fee can improve supply. Customers and workers also need to see why a price is what it is. Component 4 applies both lessons by exposing each multiplier in the API response and by limiting the total multiplier to a known maximum of 1.755 (1.30×1.35).

2.3.2 Cost-based fare structures in Sri Lanka

Nishantha studied three-wheeler services in Sri Lanka and found that fares were unregulated, varied between operators and were widely seen as unfair, with

passengers reporting overstated distances. He proposed a fare structure based on operating cost, cost recovery and a profit mark-up [20]. The Western Province Road Passenger Transport Authority now publishes a three-wheeler tariff; from 24 March 2026 it was LKR 110 for the first kilometre and LKR 90 for each additional kilometre [25]. These sources support a cost-based approach with a fixed opening charge plus a per-kilometre rate. They also provide a local benchmark for checking whether the configured fee parameters are reasonable (Section 6.2). Neither source adjusts the fare in real time for traffic, weather or fuel.

2.3.3 Machine learning for cost and price estimation

Elmousalami reviewed artificial intelligence methods for parametric construction cost estimation and found that tree ensembles and neural networks often outperform linear regression. The review also stressed that data quality and size limit accuracy [21]. Dzięcioł and Wrzesiński estimated residential building repair costs from 647 observations of 64 buildings in Warsaw using Extra Trees, Random Forest, XGBoost and gradient boosting. Their best direct regression model reached an R^2 of only 0.452 on log-transformed costs, which shows how hard repair cost prediction is with real transaction data [22]. Park and Bae compared machine learning algorithms for housing price prediction [23], and Pudaruth predicted used car prices in Mauritius with regression, k-nearest neighbours, naive Bayes and decision trees, noting that small datasets limit accuracy [24].

Table 2.1 summarises the most relevant studies and how they relate to Component 4.

Table 2.1: Summary of related research

Study	Domain and method	Key finding	Relation to Component 4
Cachon et al. [2]	Ride-hailing; analytical model of surge pricing	Surge pricing can benefit customers, workers and the platform	Supports context-dependent prices; Component 4 uses cost rather than demand
Chen and Sheldon [10]	Uber driver data	Drivers supply more hours when prices surge	Travel fee can raise technician acceptance
Rosenblat and Stark [3]	Qualitative study of Uber drivers	Opaque pricing creates information asymmetry	Itemised breakdown is returned with every fee
Brodeur and Nield [11]	NYC rides and weather shocks	Uber rides rose by roughly a fifth to a quarter in rain; fixed-fare taxis barely changed	Weather premium for rain and thunderstorms
Tsapakis et al. [4]	London travel times and weather	Rain increases urban travel time	Weather multiplier on travel effort
Nishantha [20]	Sri Lankan three-wheeler fares	Proposed cost-plus fare structure	Base fare + per-km design and local benchmark

Study	Domain and method	Key finding	Relation to Component 4
Dzięcioł and Wrzesiński [22]	Building repair costs; tree ensembles	R^2 0.452 on real repair costs	Shows difficulty of real repair data
Elmousalami [21]	Construction cost estimation review	AI methods outperform linear models; data limits accuracy	Supports tree-based estimation
Rudin [42]	Interpretable ML	Use interpretable models when accuracy is similar	Justifies single decision tree
Kapoor and Narayanan [50]	ML reproducibility	Leakage inflates reported results	Leakage disclosed and treated as a limitation

2.3.4 Evaluation practice and data leakage

Kohavi compared cross-validation and bootstrap methods for estimating accuracy and recommended stratified ten-fold cross-validation for model selection [51]. Willmott and Matsuura argued that the mean absolute error (MAE) is a more natural measure of average error than the root mean square error (RMSE) [52], while Chai and Draxler showed that RMSE is preferable when errors are normally distributed and recommended reporting both [53]. Kaufman et al. defined data leakage as the introduction of information into training that would not be available at prediction time, and described how it leads to over-estimated performance [54]. Kapoor and Narayanan surveyed leakage across 17 scientific fields and identified duplicates or near-duplicates between training and test sets as a common type [50]. These works inform the evaluation design in Chapter 3 and the treatment of the augmented dataset.

2.4 Existing systems products and techniques

Table 2.2 compares how existing platforms and practices handle the two pricing problems. The descriptions are based on the publicly visible behaviour of these services at the time of writing and on the cited literature.

Table 2.2: Pricing approaches of existing systems and practices

System / practice	Travel cost handling	Repair / service price handling	Limitation for Sri Lankan household repair
Ride-hailing (Uber, PickMe)	Algorithmic fare by distance and time, with demand-based surge	Not applicable	Designed for passenger trips; surge is demand-driven and seen as opaque [3], [19]
Three-wheeler meter tariff (WP RPTA)	Fixed first-km charge + per-km rate [25]	Not applicable	No real-time adjustment for traffic, weather or fuel
TaskRabbit-style marketplaces	Usually included in the worker's own hourly rate	Worker sets hourly rate	Customer gets no independent reference price

System / practice	Travel cost handling	Repair / service price handling	Limitation for Sri Lankan household repair
Quotation marketplaces (e.g., Thumbtack-style)	Included in quotes	Workers send quotes after the request	Price known only after quotes arrive; comparison effort for the customer
Fixed-menu home service apps	Usually a fixed visiting charge	Fixed list price per service	Prices are set by the company, not learned from local market data
Classified listings and informal referrals (common in Sri Lanka)	Negotiated by phone	Negotiated on site	No transparency; strong information asymmetry [13]
Component 4 (this work)	Cost-plus fee with live distance, traffic, weather and fuel indexation; itemised	Data-driven min–max range for worker type, service and specification	Needs real transaction data and user validation (Chapter 7)

Supporting techniques were also reviewed. When live routing is not available, road distance is often approximated by the great-circle (haversine) distance multiplied by a circuity factor. Studies have measured average circuity factors of around 1.2 to 1.4 for road networks [55]–[57]. In software architecture, microservices are independently deployable services that communicate through lightweight interfaces, which allows separate scaling and technology choices [58], [59]. REST provides a uniform interface over HTTP for such services [60]. Configuration and secrets are commonly kept in environment variables rather than code [61]. Timeouts and fallbacks protect a service from slow dependencies [62].

2.5 Gap synthesis and conceptual framework

The review shows four gaps that Component 4 addresses:

1. **Context-aware travel pricing for household technicians.** Dynamic pricing research and products focus on passenger transport and on demand-driven surge. No study was found that prices a technician's journey with a transparent cost-plus model combining live route distance, a congestion ratio, destination weather and national fuel price indexation.
2. **Local grounding.** Published Sri Lankan work on cost-based fares covers three-wheeler passenger fares [20] and does not respond to real-time conditions. Repair price data for Sri Lankan household services is not publicly available in structured form.

3. **Price ranges for household repair.** Machine learning cost studies predict single values for buildings, houses or cars [21]–[24]. None predicts a minimum–maximum range for household repair jobs from categorical descriptors aligned with a marketplace's worker categories.
4. **Deployment and explainability.** Most studies report offline accuracy only. They do not show a deployed, low-latency service whose outputs can be explained to customers and workers, as recommended by work on interpretable models [42].

Figure 2.1 presents the conceptual framework that links these gaps to the design. Two market frictions, uncompensated travel and price information asymmetry, are addressed by two mechanisms: a context-aware cost-plus transport fee and a data-driven repair price range. These are expected to increase technicians' willingness to accept jobs and customers' trust before the technician arrives, and together to improve perceived price fairness and adoption of the platform. The first two outcomes can be tested directly. The third requires the user study that is planned as future work (Section 9.4).

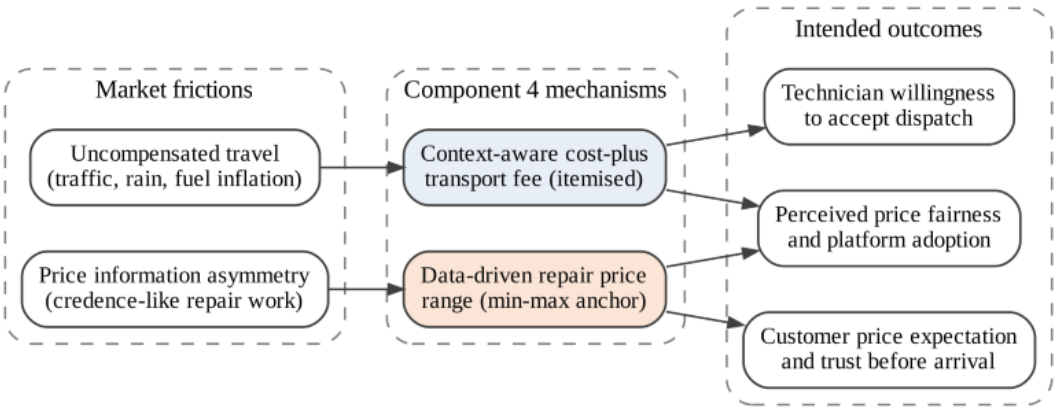


Figure 2.1: Conceptual framework of Component 4

3 RESEARCH METHODOLOGY

3.1 Research design

This research follows the Design Science Research (DSR) paradigm, in which knowledge is produced by building and evaluating an artefact that solves a relevant problem [63]. The process model of Peffers et al. was used to organise the work into six activities [64], shown in Figure 3.1. Problem identification (Chapter 1) and the literature review (Chapter 2) defined the two pricing frictions. Objectives were turned into functional and non-functional requirements (Section 4.1). The artefact, a pair of microservices, was designed and built (Chapters 4 and 5), demonstrated through deployment on Render and integration with the Flutter application, and evaluated (Chapter 6). This thesis and the group publication communicate the results.

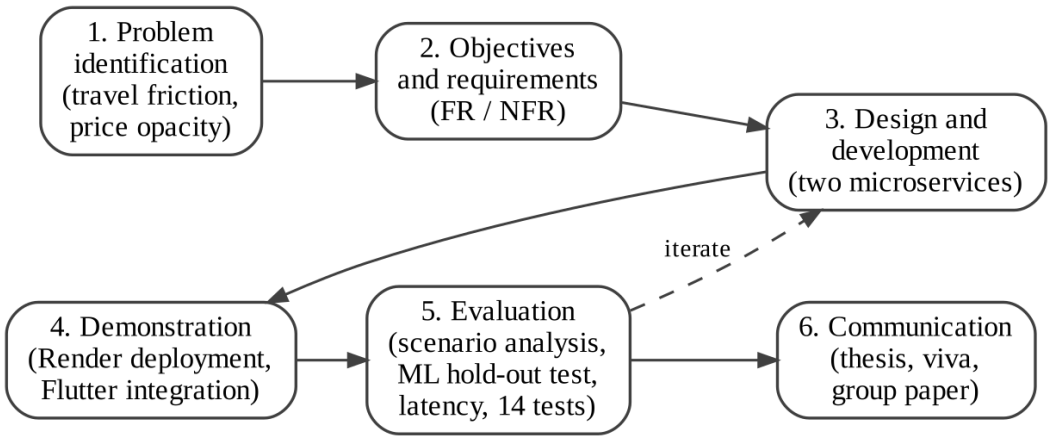


Figure 3.1: Design science research process followed in this study (adapted from Peffers et al. [64])

The two modules call for different evaluation strategies, because one is deterministic and the other is learned from data:

- **Module 1 (transport fee)** is a closed-form model. Its correctness is checked by unit and boundary tests and by recomputing documented examples. Its behaviour is studied by a *scenario analysis* that evaluates the formula over a grid of distances, traffic levels, weather conditions and the petrol prices actually published in 2026. The fee is also compared with the regulated three-wheeler tariff as a local reference.

- **Module 2 (repair price range)** is a supervised learning model. It is evaluated quantitatively on a held-out test set with MAE, RMSE and R^2 for each output, and compared with a random forest baseline.
- **Both modules** are evaluated as software services by automated tests against the deployed endpoints and, for Module 2, by measuring inference latency.

The research is therefore mainly quantitative and experimental, combined with design-oriented reasoning about transparency and explainability. User-perceived fairness and trust, which belong to the conceptual framework in Figure 2.1, were not measured in this phase. Section 6.3 describes the planned study.

3.2 Data participants or study materials

3.2.1 Repair price dataset

The repair price dataset was built for this research because no structured dataset of Sri Lankan household repair prices was available. Price information was collected through a direct field survey of independent tradespeople and local repair workshops in Sri Lanka. They were asked for typical labour charges, the cost of common replacement parts where these are normally included in the job, and quotations for typical jobs. The figures were then cross-checked against prices published on Sri Lankan repair service directories and platforms to make sure each range was realistic.

For each job type, the collected information was summarised as one *base combination* consisting of a worker type, a service, a specification and two prices. The minimum price (`min_price`) represents a typical simple job of that type, mainly labour. The maximum price (`max_price`) represents a more complex job, for example one needing component replacement or difficult access. In total, 105 base combinations were defined. The structure of the dataset is given in Table 3.1, and examples are shown in Table 3.2.

Table 3.1: Structure of the repair price dataset

Field	Type	Description	Cardinality
worker_type	Categorical (input)	Trade, using the same labels as Component 1 (e.g., AC technician, plumber, painter)	39 categories
service	Categorical (input)	Task within the trade (e.g., AC installation, tap / faucet replacement)	95 distinct services
specification	Categorical (input)	Size, capacity or job unit (e.g., 12,000 BTU inverter; 55 inch; per 100 sq ft, 2 coats)	91 distinct specifications
min_price	Numeric (target)	Lower bound of the typical job price in LKR	Rounded to LKR 500; ≥ 1,000
max_price	Numeric (target)	Upper bound of the typical job price in LKR	Rounded to LKR 500; ≥ min_price + 1,000

Table 3.2: Representative trade, service and specification combinations in the dataset

Trade (worker_type)	Task (service)	Sizing / specification
AC Technician	AC Servicing / General Cleaning	12,000 BTU Inverter; 18,000 – 24,000 BTU
AC Technician	AC Installation	Split Type (9,000 – 12,000 BTU)
Electronics Technician	Display Replacement	32 inch; 42 inch; 55 inch; 65 inch
Electronics Technician	Backlight Replacement	32 inch – 43 inch; 50 inch – 65 inch
Aluminium Fabricator	Aluminium Window Fabrication	Casement / Sliding (per sq ft)
Plumber	Tap / Faucet Replacement	Standard Tap Replacement
Painter	Interior Wall Painting	Per 100 Sq Ft (2 Coats)
Carpenter	Door Installation / Fitting	Solid Wood Door Fitting

The specification field is what makes the estimate useful. A 65-inch television display replacement and a 32-inch one belong to the same trade and service but have very different costs. Because the specification is expressed in the units customers already use (BTU, inches, square feet), the customer can select it without technical knowledge.

3.2.2 Data augmentation

One hundred and five rows are too few to train and test a model, and real prices vary between regions and between technicians. Each base combination was therefore expanded into six rows, giving 630 rows, using the rules in Table 3.3. The aim was to represent realistic local variation around each surveyed range, not to create new kinds of jobs. Adding small random noise to training inputs or targets is a known

form of regularisation [65], but it does not add new information about unseen combinations.

Table 3.3: Augmentation rules applied to each base combination

Step	Rule	Purpose
1. Replication	Each of the 105 base combinations is expanded into 6 rows (105 × 6 = 630)	Enough rows for training and testing
2. Noise	min_price and max_price each multiplied by a factor drawn from a uniform distribution on [0.92, 1.08] (±8%)	Represent regional and technician price variation
3. Rounding	Each value rounded to the nearest LKR 500: round(price / 500) × 500	Match common Sri Lankan billing practice
4. Floor	min_price constrained to at least LKR 1,000	Avoid unrealistic very low prices
5. Spread	max_price constrained to at least min_price + LKR 1,000	Keep a positive, meaningful range

Rounding to LKR 500 interacts with the noise. For low-priced jobs, ±8% is often smaller than half the rounding step, so most augmented rows round back to the original value. For expensive jobs, for example a large display replacement, ±8% spans several rounding steps, so augmented rows differ by thousands of rupees. Most of the variation in the targets therefore comes from the expensive jobs, which affects how the error metrics should be read (Section 6.1).

3.2.3 Data for the transport fee engine

The transport fee engine does not need a training dataset. Its inputs at run time are the origin and destination coordinates. From these it obtains route distance, normal duration and duration in traffic from the Google Maps Distance Matrix API [26], and the current weather condition at the destination from the OpenWeatherMap Current Weather API [27]. The petrol price series used for the scenario analysis was taken from published CEYPETCO revisions in 2026 [7]–[9], and the three-wheeler reference tariff from the Western Province Road Passenger Transport Authority announcement of March 2026 [25].

3.2.4 Participants

The only human participants in this phase were the tradespeople and workshop owners who provided price information in the field survey. The dataset stores only aggregated job prices and job descriptors, and no names, contact details or locations

of the respondents. No customers or technicians took part in a usability or acceptance study; this is planned as future work (Section 6.3).

3.3 Experimental or study procedure

3.3.1 Repair price model experiment

The experiment followed the pipeline in Figure 3.2:

1. The augmented 630-row dataset was loaded with pandas.
2. The three categorical inputs (worker_type, service, specification) were separated from the two targets (min_price, max_price).
3. The rows were split at random into a training set (80%) and a test set (20%) with scikit-learn's `train_test_split` (test_size = 0.20).
4. A `ColumnTransformer` containing a `OneHotEncoder` with `handle_unknown='ignore'` was fitted on the training inputs only and then applied to the test inputs, so that the encoder learned nothing from the test set.
5. A multi-output `DecisionTreeRegressor` with `max_depth = 12` was trained on the encoded training data to predict both targets together.
6. A multi-output `RandomForestRegressor` with 100 trees was trained on the same data as a baseline.
7. Both models predicted the test set, and MAE, RMSE and R^2 were computed separately for min_price and max_price.
8. The decision tree pipeline (encoder plus model) was serialised with joblib for deployment.

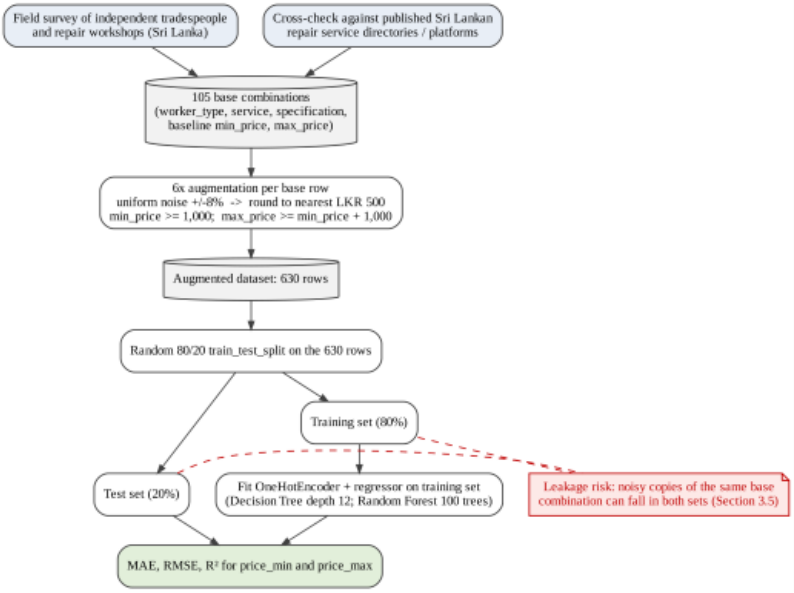


Figure 3.2: Dataset construction and evaluation pipeline for the repair price model

3.3.2 Transport fee scenario analysis

The transport fee formula (Section 4.4.1) was evaluated with the configured parameters for route distances from 0 to 20 km. Four representative conditions were used: normal traffic in clear weather, moderate traffic in clear weather, heavy traffic in rain, and heavy traffic during a thunderstorm. A second analysis held distance at 10 km and conditions at normal and clear, and varied the current petrol price over the six values published or configured in 2026. The documented API example (5.4 km, heavy traffic, rain) was decomposed into its components to check the itemised breakdown. Because the fee is deterministic, these analyses describe exactly what the deployed service returns for the given inputs. They do not depend on live API data.

3.3.3 Software verification and latency measurement

Automated tests were written with pytest for the calculation logic and input validation, and a Postman collection was run against the live Render endpoints (Section 5.4). The inference latency of the deployed decision tree pipeline was measured with Python's `time.perf_counter()` around repeated estimation requests on Render, and the mean, minimum and maximum were recorded. The measurement covers processing inside the service. It does not include the mobile network round trip between the phone and the Render data centre, which depends on the user's connection.

3.4 Measures and evaluation metrics

For each target (min_price and max_price), with n test rows, true values y_i, predictions $\hat{y}_i$ and mean of the true values $\bar{y}$, the following metrics were calculated [38], [52], [53]:

$$\text{MAE} = (1/n)\ \Sigma\ |y_i - \hat{y}_i| \tag{3.1}$$

$$\text{RMSE} = \sqrt{[\ (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2\ ]} \tag{3.2}$$

$$R^2 = 1 - \Sigma\ (y_i - \hat{y}_i)^2\ /\ \Sigma\ (y_i - \bar{y})^2 \tag{3.3}$$

MAE gives the average error in rupees and is easy to explain to non-specialists. RMSE weights large errors more heavily, so a large gap between RMSE and MAE indicates that a few predictions are far off. R^2 gives the share of variance in the target explained by the model, where 1.0 is perfect and 0 is no better than always predicting the mean. The other evaluation measures are summarised in Table 3.4.

Table 3.4: Evaluation measures used in this study

Aspect	Measure	Target / criterion
Repair model accuracy	MAE, RMSE, R^2 per output on the 20% hold-out set	Comparison with the random forest baseline
Range validity	price_min ≤ price_max for every prediction	Always true (tested)
Repair service latency	Mean, minimum and maximum inference time (ms)	Mean below 15 ms
Transport fee correctness	Recalculation of documented examples; unit tests	Exact match after rounding
Transport fee behaviour	Fee across distance, traffic, weather and fuel scenarios	Monotonic in each factor; capped multiplier 1.755
Robustness	Boundary tests (invalid or missing coordinates, timeouts, unseen categories)	Correct error response or fallback
Overall quality	14 automated tests (8 transport, 6 ML)	100% pass

No fixed accuracy threshold was set in advance; the random forest serves as the reference point for accuracy. The 15 ms latency target comes from the component specification, which aimed for near-instant updates of the estimate in the mobile interface. It is a practical engineering goal rather than a value taken from the literature.

3.5 Data analysis

Results for the two models were compared output by output. Because only a single random split was available, no confidence intervals or significance tests could be calculated, so small differences between the models are interpreted with caution.

3.5.1 Data leakage in the evaluation design

The split was made on the 630-row augmented dataset, after augmentation. Each base combination appears six times with small price differences. A random 80/20 split therefore almost always places some copies of a combination in the training set and others in the test set. If each test row has five siblings and each sibling falls in the training set with probability of about 0.8, the chance that none of them is in training is about $0.2^5 \approx 0.0003$. In practice, almost every test row has at least one near-duplicate in training. This is a recognised form of data leakage [50], [54]: the test set does not contain genuinely new jobs, so the metrics measure how well the model reproduces known combinations with noise, not how well it would price a new combination.

The results of the original experiment are reported unchanged, and this limitation is stated openly rather than hidden. The results in Chapter 6 are therefore labelled as optimistic, and Chapter 7 discusses what they do and do not show. A grouped split, in which all six copies of a base combination stay on the same side, is recommended as the first item of future work (Section 9.4).

3.5.2 Analysis of the transport fee

The scenario outputs were analysed by checking monotonicity (the fee should never fall when distance, congestion, weather severity or fuel price rises), the size of each premium in rupees, and the relation to the three-wheeler reference tariff. Because the formula is a product of linear terms and multipliers, these properties can also be shown analytically (Section 4.4.1).

3.6 Ethics privacy security and responsible AI

Privacy. The transport service receives two pairs of coordinates per request: the technician's current location and the customer's home. These are personal data under Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [66]. The service is stateless

and does not store coordinates; it forwards them only to Google Maps and OpenWeatherMap to obtain route and weather data. The repair estimator receives only job descriptors and no personal data.

Security. All third-party API keys and the fuel price variables are stored as server-side environment variables on Render and never reach the mobile application [61]. Requests are validated before external calls are made, which limits the misuse of paid APIs. Measured against the OWASP API Security Top 10 [67], the main remaining risks are the lack of authentication and of rate limiting on the public endpoints in the current development stage (Section 5.5).

Fairness and transparency. The transport fee uses only trip conditions. It does not use customer identity, neighbourhood or willingness to pay, and it does not respond to demand. The same trip in the same conditions always receives the same fee, and each factor is shown in the breakdown. The repair estimate is labelled as an indicative range, not a binding quotation, so that technicians can still price unusual jobs. The total multiplier is capped by design at 1.755, which prevents extreme prices in severe conditions.

Responsible reporting. The leakage risk and the absence of a user study are disclosed in this chapter, in Chapter 6 and in Section 7.5. The use of generative AI tools in preparing this thesis is disclosed in Appendix B.

3.7 Reproducibility and research artefacts

The research artefacts are listed in Table 3.5. Fee parameters, multiplier thresholds and model hyperparameters are stated in full in this thesis so that the calculations can be repeated. The random seed used for the train/test split should be recorded with the code repository so that the exact split can be reproduced.

Table 3.5: Research artefacts produced in this study

Artefact	Description
Repair price dataset	105 base combinations and the 630-row augmented CSV (worker_type, service, specification, min_price, max_price)
Training script	Data split, encoding, decision tree and random forest training, metric calculation, latency benchmark
Serialised model	joblib file containing the ColumnTransformer / OneHotEncoder and the multi-output decision tree

Artefact	Description
Transport fee service	Flask application with Gunicorn configuration and environment-variable based parameters
Repair estimator service	FastAPI application with Uvicorn and a Dockerfile
Test suite	pytest unit tests (test_app.py) and a Postman collection for the live endpoints
Deployment	Two Render web services with environment variables for keys and fuel prices

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the problem statement, the objectives in Table 1.1, the needs of the other components and discussions within the group. Functional requirements (FR) are listed in Table 4.1 and non-functional requirements (NFR) in Table 4.2. The NFRs are grouped using the quality characteristics of ISO/IEC 25010 [68].

Table 4.1: Functional requirements

ID	Requirement	Module
FR1	Accept origin (technician) and destination (customer) coordinates and validate their ranges and presence.	1
FR2	Obtain route distance, normal duration and duration in traffic for the trip from the Google Maps Distance Matrix API.	1
FR3	Obtain the current weather condition at the destination from OpenWeatherMap.	1
FR4	Calculate the transport fee as a base fare plus a fuel-indexed distance cost, multiplied by traffic and weather multipliers, rounded to the nearest rupee.	1
FR5	Return the total fee with an itemised breakdown (base fare, distance cost, traffic multiplier, weather multiplier, fuel ratio) and the trip context (distance, delay, weather).	1
FR6	Read fare parameters, petrol prices and API keys from server configuration so that they can be changed without code changes.	1
FR7	Accept a trade, task and sizing for a repair job and return a predicted minimum and maximum price in LKR.	2
FR8	Use worker categories that match Component 1's 39 labels.	2
FR9	Return a valid range (price_min ≤ price_max) for every accepted request, including categories not seen in training.	2
FR10	Expose both functions as REST endpoints that the Flutter application can call over HTTPS.	1, 2

Table 4.2: Non-functional requirements

ID	Quality characteristic	Requirement
NFR1	Performance efficiency	Mean server-side inference time of the repair estimator below 15 ms; transport fee returned within the time of the two external API calls plus negligible computation.
NFR2	Reliability (fault tolerance)	If the weather service times out, the fee is still returned with a neutral weather multiplier of 1.00. If routing fails, a clear error is returned instead of a wrong fee.
NFR3	Security	API keys and fuel prices are kept only on the server as environment variables; the mobile application never holds them.

ID	Quality characteristic	Requirement
NFR4	Maintainability	Each module is an independent service that can be changed, redeployed and scaled without affecting the other or the rest of the platform.
NFR5	Usability / transparency	Every fee is accompanied by its breakdown; every repair estimate is a range in LKR that can be shown directly in the user interface.
NFR6	Compatibility	JSON request and response formats with stable field names, usable by Flutter now and by the Node.js gateway later.
NFR7	Portability	Services run in stateless Linux containers on a platform-as-a-service provider.

4.2 Component architecture within the complete solution

Figure 1.1 in Chapter 1 showed Component 4 in the context of the whole marketplace. The Flutter application talks to the Node.js backend for accounts, bookings and the services of Components 1 to 3. Component 4 receives two kinds of input from the other components. The worker type comes from Component 1, which uses the same 39 labels as the repair dataset. The selected technician's GPS coordinates come from Component 2's recommendation results. In the current development stage, the Flutter application calls Component 4's two endpoints directly. This allowed the component to be developed and tested independently of the other team members' work. In the target production architecture, these calls will be routed through the Node.js API gateway, which will add JSON Web Token (JWT) authentication, attach booking data and combine the responses.

Figure 4.1 shows the internal architecture of Component 4. The two modules are separate microservices [58], [59] on Render, a platform-as-a-service (PaaS) provider. Module 1 is a synchronous Flask application served by Gunicorn, a WSGI server [29]. It depends on two external APIs and reads its parameters from environment variables. Module 2 is an asynchronous FastAPI application served by Uvicorn, an ASGI server [28], packaged as a Docker container [69]. It loads the serialised model pipeline into memory at start-up.

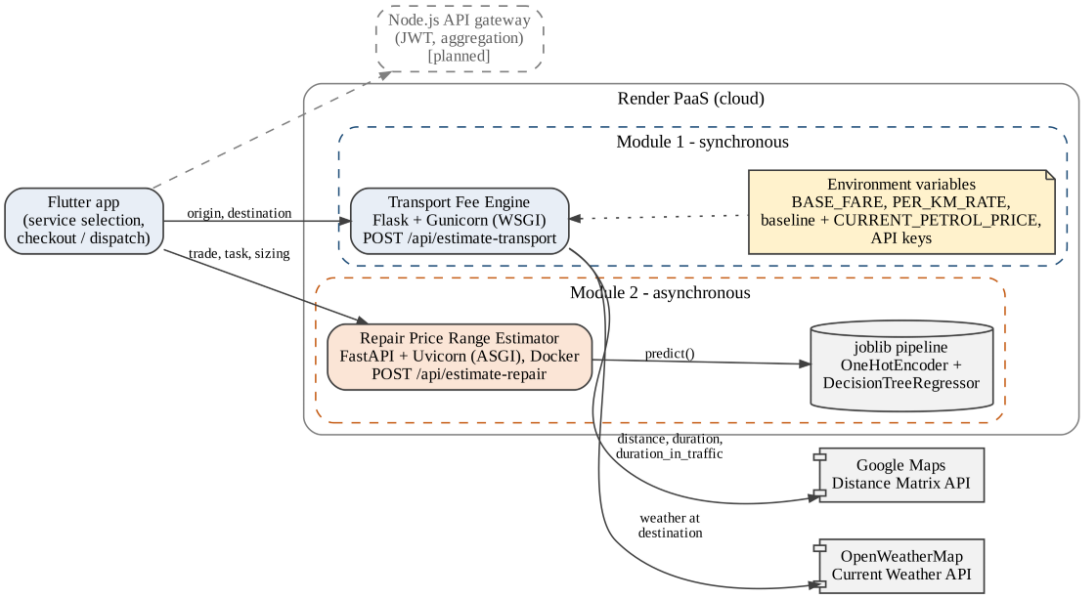


Figure 4.1: Internal architecture and deployment of Component 4

4.3 Design alternatives and decisions

The main design decisions and the alternatives considered are summarised in Table 4.3.

Table 4.3: Design alternatives and decisions

Decision	Alternatives considered	Choice and rationale
Transport pricing approach	(a) Flat call-out charge; (b) demand-based surge as in ride-hailing; (c) context-aware cost-plus	(c): cost-based increases are perceived as fair [15], [16], are explainable item by item, and do not exploit high demand. (a) ignores real cost differences; (b) is opaque and controversial [3], [19].
Distance source	(a) Straight-line (haversine) distance × circuity factor; (b) live routing API	(b) Google Distance Matrix: real road distance and traffic durations in one call [26]. (a) cannot measure congestion; kept as a future offline fallback [56].
Traffic adjustment	(a) Continuous multiplier equal to the traffic ratio; (b) discrete levels	(b) three levels (1.00/1.15/1.30): easy to explain ("heavy traffic"), stable against small fluctuations, bounded.
Fuel adjustment	(a) Fixed per-km rate; (b) automatic scraping of CEYPETCO prices; (c) manually updated environment variable	(c): prices change by gazette at most a few times a month [7], [9]; a variable avoids fragile scraping and needs no redeployment. (b) is future work.
Repair model	Linear regression; k-nearest neighbours; quantile regression; random forest; single decision tree	Multi-output decision tree: comparable accuracy to random forest (Chapter 6), interpretable as rules [42], small, fast, and predicts both bounds from the same leaf [40].
Range method	(a) Quantile regression of individual prices; (b) direct prediction of min and max	(b): the survey recorded minimum and maximum quotations, not individual transactions, so quantiles could not be estimated [47].
Service framework	Single framework for both; Flask; FastAPI	Flask for the simple synchronous I/O-bound fee calculation; FastAPI with ASGI for the estimator, which is called repeatedly while users change selections.

Decision	Alternatives considered	Choice and rationale
Architecture	Add pricing to the Node.js monolith; two separate microservices	Microservices: Python ML stack, independent deployment, failure isolation and separate scaling [59].
Secrets handling	Keys in the mobile app; keys on the server	Server-side environment variables [61]; keys can be rotated without an app release.

A machine learning model was deliberately **not** used for the transport fee. A learned fee would need historical data on accepted and rejected jobs at different prices, which a new platform does not have. It would also be harder to explain to technicians and customers. A transparent formula with configurable parameters suits the platform's current stage. It can later be calibrated with data on job acceptance.

4.4 Detailed design

4.4.1 Transport fee model

The transport fee is defined by Equation (4.1):

$$TotalFee = \text{round}([\ B + D \times r_d \times (F_c / F_b)\ ] \times M_{traffic} \times M_{weather}\ ) \qquad (4.1)$$

The symbols and configured values are given in Table 4.4. The term in square brackets is the cost-plus *subtotal*: a fixed call-out overhead plus a distance cost. The distance cost is indexed to fuel through the ratio of the current petrol price to a baseline price. When petrol becomes more expensive, the per-kilometre rate rises in proportion without any change to the code. The two multipliers then add premiums for difficult travel conditions.

Table 4.4: Parameters of the transport fee model

Symbol	Meaning	Value / source
B	Base fare: fixed call-out overhead	LKR 200 (BASE_FARE)
D	Road distance from technician to customer	Google Maps Distance Matrix API, km
r_d	Operational distance rate	LKR 80 per km (PER_KM_RATE)
F_b	Baseline petrol price (reference before recent increases)	LKR 350 per litre
F_c	Current CEYPETCO Petrol 92 Octane price	LKR 414 per litre from 1 October 2026 [9] (CURRENT_PETROL_PRICE)
F_c/F_b	Fuel index ratio	$414 / 350 \approx 1.183$ (+18.3%)
$M_{traffic}$	Traffic multiplier	1.00, 1.15 or 1.30 (Table 4.5)
$M_{weather}$	Weather multiplier	1.00, 1.20 or 1.35 (Table 4.6)

The base fare and distance rate were chosen to be close to local transport costs. The Western Province three-wheeler tariff from March 2026 is LKR 110 for the first kilometre and LKR 90 for each further kilometre [25]. The configured LKR 200 call-out charge is higher than a passenger's first-kilometre fare because it also covers the technician's time to prepare and carry tools. The LKR 80 per km rate, before fuel indexation, is slightly below the three-wheeler per-km fare.

Traffic multiplier. The traffic ratio is computed from the Distance Matrix response as in Equation (4.2), which is the same idea as the Travel Time Index [35]:

$$ratio = duration_in_traffic \,/\, duration \tag{4.2}$$

The ratio is mapped to three levels as in Table 4.5 and Figure 4.2. The thresholds of 15% and 40% delay separate ordinary variation from noticeable congestion and from gridlock.

Table 4.5: Traffic multiplier rules

Condition	Traffic ratio	Delay over normal time	M_traffic
Normal flow	< 1.15	< 15%	1.00
Moderate congestion	1.15 ≤ ratio < 1.40	15% – 39%	1.15
Heavy gridlock	≥ 1.40	≥ 40%	1.30

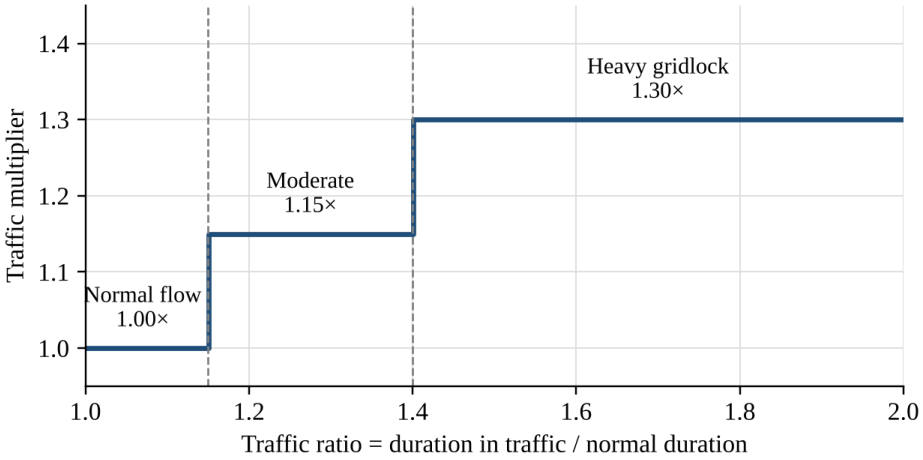


Figure 4.2: Step function mapping the traffic ratio to the traffic multiplier

Weather multiplier. OpenWeatherMap reports a main weather group for each location [27]. The engine maps it as in Table 4.6. Thunderstorms receive the highest premium because riding a motorcycle or three-wheeler in lightning and heavy rain is

dangerous. If the weather call times out or fails, the multiplier is set to 1.00 so that the customer is never charged for a premium that could not be confirmed.

Table 4.6: Weather multiplier rules

OpenWeatherMap condition at destination	M_weather	Rationale
Rain or drizzle	1.20	Slower travel and discomfort [4], [5]; keeps supply available [11]
Thunderstorm	1.35	Safety risk for two- and three-wheel vehicles
All other conditions (clear, clouds, mist, etc.)	1.00	No premium
Service timeout or error	1.00	Neutral fallback; never charge an unconfirmed premium

Properties of the model. Several useful properties follow directly from Equation (4.1):

- *Floor.* When D approaches zero in normal conditions, the fee approaches the base fare of LKR 200. A technician is never asked to travel for nothing.
- *Linearity in distance.* For fixed conditions the fee rises linearly with distance, with a slope of $r_d \times (F_c/F_b) \times M_{traffic} \times M_{weather}$. At the current fuel ratio, this is about LKR 94.6 per km in normal conditions and LKR 166.1 per km in the worst case.
- *Monotonicity.* The fee never falls when distance, fuel price, congestion level or weather severity rises.
- *Bounded premium.* The combined multiplier is at most $1.30 \times 1.35 = 1.755$, so the fee in the worst conditions is at most 75.5% above the fee for the same trip in normal conditions. There is no other ceiling. Requests with invalid coordinates are rejected at validation.
- *Determinism.* The same inputs and conditions always produce the same fee, which makes disputes easy to check.

4.4.2 Transport fee processing flow

Figure 4.3 shows the request processing flow. Validation happens first so that no paid API call is made for invalid input. Routing failure is treated as an error because the distance is essential. Weather failure is handled by the fallback because the

weather premium is optional. This follows the timeout-and-fallback pattern for integration points recommended by Nygard [62].

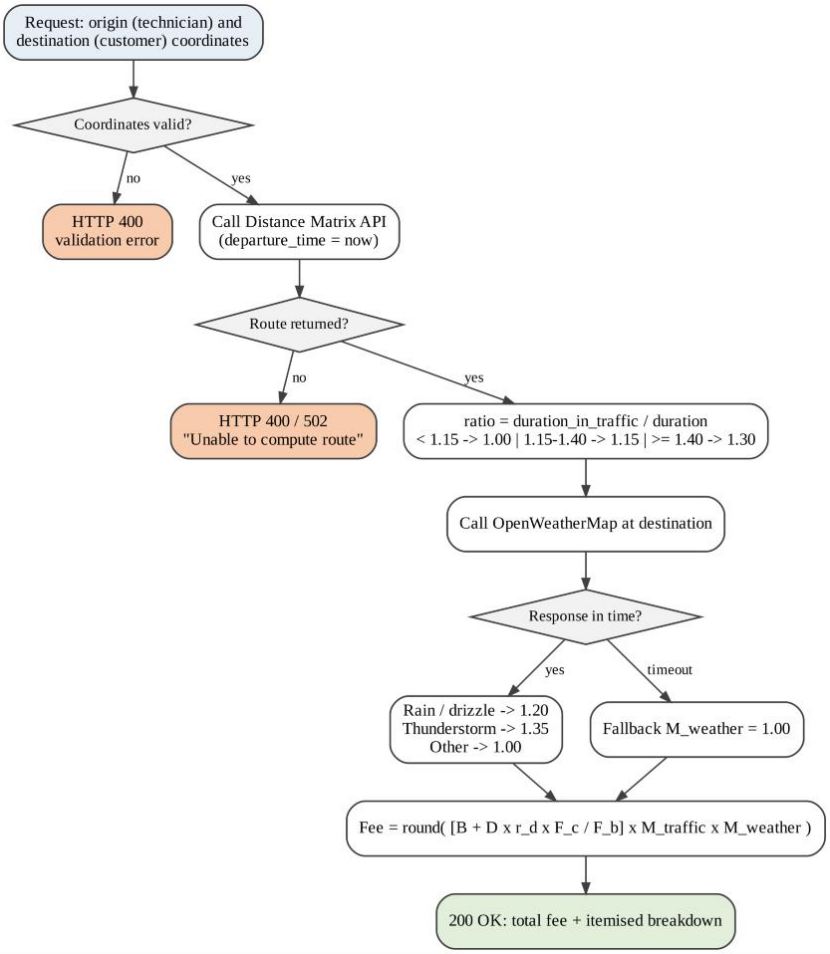


Figure 4.3: Processing flow of the transport fee engine

4.4.3 Transport fee API contract

The transport service exposes one endpoint. Listing 4.1 shows a request and the corresponding response, using the example from the component specification.

Listing 4.1: Request and response of POST /api/estimate-transport

```
POST /api/estimate-transport
{
  "origin":      { "lat": 6.9344, "lng": 79.8428 },
  "destination": { "lat": 6.8883, "lng": 79.8588 }
}

200 OK
{
  "distanceKm": 5.4,
  "trafficDelayMins": 7,
  "weatherCondition": "Rain",
  "totalTransportFee": 1109,
  "breakdown": {
    "baseFare": 200.0,
    "distanceCost": 511.06,
    "trafficMultiplier": 1.30,
    "weatherMultiplier": 1.20,
    "fuelRatio": 1.183
  }
}
```

The example can be checked by hand: the distance cost is 5.4 × 80 × 1.183 = 511.06, the subtotal is 200 + 511.06 = 711.06, and the fee is 711.06 × 1.30 × 1.20 = 1,109.25, which rounds to LKR 1,109. Because every intermediate value is returned, the mobile application can show the customer exactly why the fee is LKR 1,109 rather than LKR 711.

4.4.4 Repair price model

The repair estimator is a scikit-learn pipeline [41] with two stages, shown in Figure 4.4. The `ColumnTransformer` applies a `OneHotEncoder` to the three categorical inputs, producing a sparse binary vector with one column per known category value. Setting `handle_unknown='ignore'` means that a value not seen in training is encoded as all zeros for that field instead of causing an error. The encoded vector goes to a `DecisionTreeRegressor` with two targets, min_price and max_price, and a maximum depth of 12.

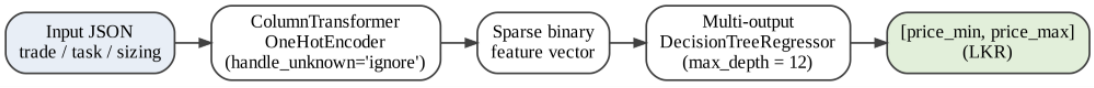


Figure 4.4: Repair price estimation pipeline

A regression tree splits on one binary feature at a time, for example “service = Display Replacement”, and each leaf predicts the mean min_price and mean max_price of the training rows that reach it [36]. Because both bounds come from the same leaf, they always describe the same group of similar jobs. Because each

bound is a mean of training values that themselves satisfy min ≤ max, the predicted minimum can never exceed the predicted maximum. This was confirmed by the tests in Section 5.4.

The single tree was preferred to an ensemble for three reasons. First, its partitions do not overlap: every prediction comes from exactly one leaf, a single group of training rows defined by explicit rules, while a forest averages the leaves of many trees grown on different bootstrap samples, which may group different jobs. The component specification describes this blending as “cross-category smoothing”. Second, each prediction can be explained as a short path of rules, which supports the transparency goal [42], [43]. Third, a single tree is small and fast to evaluate. The depth limit of 12 controls the size of the tree. It also means that some leaves may group more than one service, which is examined in Section 7.1.

4.4.5 Repair estimate API contract

The estimator exposes one endpoint, shown in Listing 4.2. The response repeats the request fields so that the client can match responses to the user’s latest selection when several requests are in flight, which is one reason for using an asynchronous framework.

Listing 4.2: Request and response of POST /api/estimate-repair (example from the API specification)

```
POST /api/estimate-repair
{
  "trade": "Plumbing",
  "task":  "Pipe Leak Repair",
  "sizing": "Moderate"
}

200 OK
{
  "trade": "Plumbing",
  "task": "Pipe Leak Repair",
  "sizing": "Moderate",
  "price_min": 2500,
  "price_max": 4200,
  "currency": "LKR"
}
```

4.4.6 Interaction design

Figure 4.5 shows how the two endpoints are used in a booking. The repair estimate is requested as soon as the customer has a service and specification, so it appears while the customer is still deciding. The transport fee is requested only after a technician is

chosen, because it depends on that technician's location. At confirmation the customer sees both figures: an indicative repair range and an exact, itemised transport fee.

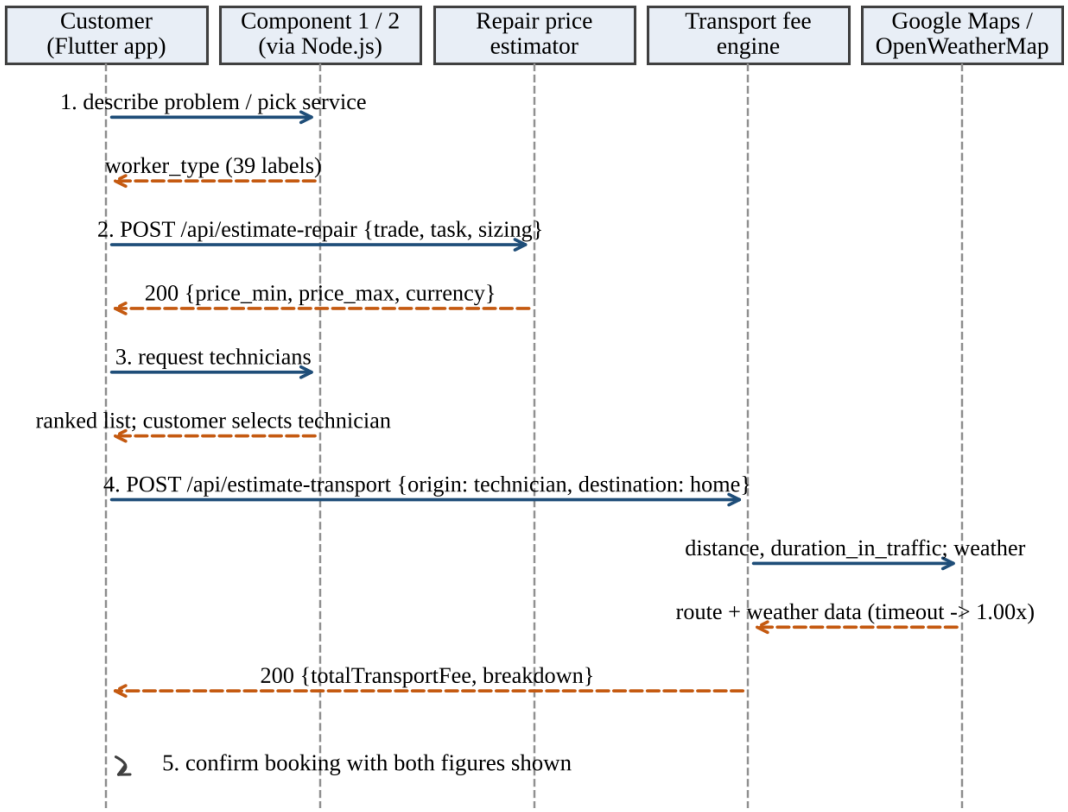


Figure 4.5: Sequence of pricing interactions during a booking

4.4.7 Error handling design

Table 4.7 lists the error and fallback behaviour of both services.

Table 4.7: Error handling and fallback behaviour

Situation	Service	Behaviour
Missing origin or destination, or coordinates out of range (e.g., latitude > 90)	Transport	HTTP 400 with a validation message; no external call
Google Maps returns non-200 status or ZERO_RESULTS	Transport	HTTP 400 or 502 with "Unable to compute route between coordinates"
OpenWeatherMap timeout or error	Transport	M_weather = 1.00; fee returned normally
Identical origin and destination (zero distance)	Transport	Handled without error; the distance cost is zero, so the fee reduces to the base fare times any multipliers
Missing trade key	Repair	Request rejected with a validation error
Trade, task or sizing not seen in training	Repair	Encoded as zeros (handle_unknown='ignore'); a range is still returned

4.5 Standards and professional practice

The design follows these standards and practices:

- **ISO/IEC 25010** product quality model, used to structure the non-functional requirements [68].
- **REST** architectural style and **JSON** message formats for stateless, cacheable and uniform interfaces [60].
- **The Twelve-Factor App** principles: configuration in the environment, stateless processes and disposable services [61].
- **ISO/IEC/IEEE 29119** concepts for test design, including boundary value testing and the separation of unit and system-level tests [70].
- **OWASP API Security Top 10** as a reference for API risks such as missing authentication and unrestricted resource consumption [67].
- **Sri Lanka Personal Data Protection Act, No. 9 of 2022**, for handling location data [66], and the **Consumer Affairs Authority Act, No. 9 of 2003**, which sets expectations about fair trading and price information for consumers [71].
- **IEEE referencing** and disclosure of AI tool use, as required by the SLIIT thesis guidelines.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Both services were implemented in Python 3.11. The technology stack is summarised in Table 5.1. Python was chosen because the machine learning libraries used by the group are Python-based. It also let the transport and repair services share development practices even though they use different web frameworks.

Table 5.1: Technology stack of Component 4

Layer	Module 1: transport fee engine	Module 2: repair price estimator
Language	Python 3.11	Python 3.11
Web framework	Flask, flask-cors	FastAPI
Server	Gunicorn (WSGI)	Uvicorn (ASGI)
Main libraries	requests (HTTP calls to external APIs)	scikit-learn, pandas, joblib
External services	Google Maps Distance Matrix API; OpenWeatherMap Current Weather API	None
Packaging	Stateless Linux web service	Docker container
Hosting	Render (PaaS)	Render (PaaS)
Configuration	Environment variables: base fare, per-km rate, baseline and current petrol price, API keys	Model file bundled with the service
Testing	pytest; Postman	pytest; Postman
Client	Flutter mobile application (HTTP POST)	Flutter mobile application (HTTP POST)

5.2 Component implementation

5.2.1 Transport fee engine

The Flask application defines one route, `POST /api/estimate-transport`. On start-up it reads `BASE_FARE`, `PER_KM_RATE`, the baseline petrol price, `CURRENT_PETROL_PRICE` and the two API keys from the environment, so that no secret or tariff value is written in the code. Listing 5.1 gives the processing logic in pseudocode.

Listing 5.1: Pseudocode of the transport fee calculation

```
function estimate_transport(request):
    origin, destination <- request.origin, request.destination
    if origin or destination missing, or lat not in [-90, 90],
       or lng not in [-180, 180]:
        return 400, "invalid coordinates"

    route <- DistanceMatrix(origin, destination, departure_time = now)
    if route.status != OK or route has ZERO_RESULTS:
        return 400/502, "Unable to compute route between coordinates"
    D      <- route.distance_m / 1000
    ratio  <- route.duration_in_traffic / route.duration
    M_traffic <- 1.30 if ratio >= 1.40
                 else 1.15 if ratio >= 1.15 else 1.00

    try:
        weather <- OpenWeatherMap(destination, timeout)
        M_weather <- 1.35 if weather is Thunderstorm
                     else 1.20 if weather in {Rain, Drizzle} else 1.00
    except timeout or error:
        M_weather <- 1.00                       # neutral fallback

    fuel_ratio    <- CURRENT_PETROL_PRICE / BASELINE_PETROL_PRICE
    distance_cost <- D * PER_KM_RATE * fuel_ratio
    subtotal      <- BASE_FARE + distance_cost
    total_fee     <- round(subtotal * M_traffic * M_weather)
    return 200, {distanceKm, trafficDelayMins, weatherCondition,
                 totalTransportFee: total_fee, breakdown{...}}
```

The delay in minutes reported to the client is the difference between the duration in traffic and the normal duration. For example, a 7-minute delay on a trip that normally takes 15 minutes gives a ratio of about 1.47 and therefore the heavy-traffic multiplier, as in the API example. The final fee is rounded to the nearest whole rupee with Python's built-in `round()` function, so the customer always sees an integer amount.

CORS is enabled with flask-cors so that web-based test clients can call the service during development. Gunicorn serves the Flask application in production. Because each request is independent and most of the time is spent waiting for the external APIs, the service can scale horizontally by adding Gunicorn workers or service instances.

5.2.2 Repair price model training

The training script reads the 630-row CSV with pandas, separates the three input columns from the two target columns, and splits the data 80/20 with `train_test_split`. It then builds the pipeline described in Section 4.4.4. The encoder is fitted only on the training data. After the evaluation in Section 3.3, the

fitted decision tree pipeline is saved with joblib. Listing 5.2 gives the procedure in pseudocode.

Listing 5.2: Pseudocode of the repair model training and evaluation

```
data <- read_csv("repair_prices_630.csv")
X <- data[worker_type, service, specification]
Y <- data[min_price, max_price]
X_train, X_test, Y_train, Y_test <- train_test_split(X, Y, test_size =
0.20)

encoder <- ColumnTransformer(OneHotEncoder(handle_unknown = 'ignore'))
dt <- Pipeline(encoder, DecisionTreeRegressor(max_depth = 12))
rf <- Pipeline(encoder, RandomForestRegressor(n_estimators = 100))
fit dt and rf on (X_train, Y_train)        # encoder fitted on training
data only

for model in {dt, rf}:
    P <- model.predict(X_test)
    for target in {min_price, max_price}:
        report MAE, RMSE, R2 (Y_test[target], P[target])

benchmark latency of dt.predict on single requests
joblib.dump(dt, "repair_price_model.joblib")
```

5.2.3 Repair price estimator service

The FastAPI application loads the joblib pipeline once at start-up and keeps it in memory, so no model loading happens during a request. The endpoint `POST /api/estimate-repair` validates the request body, builds a one-row data frame with the trade, task and sizing, calls `predict`, and returns the two predicted bounds with the currency code. FastAPI validates the request body against the declared fields, so a request without the trade key is rejected with a validation error before it reaches the model. The service is packaged with a Dockerfile that installs the Python dependencies and starts Uvicorn.

5.3 Integration with the other components

Integration happens at three points:

- **Component 1 → Component 4 (worker type).** The repair dataset uses the same 39 worker categories that Component 1 predicts from the customer's description. Once Component 1 has identified, for example, an AC technician, the Flutter application can offer the services and specifications available for that trade and request a repair estimate without any translation between label sets.

- **Component 2 → Component 4 (technician location).** After the customer selects a technician from Component 2's recommended list, the application takes the technician's current GPS coordinates (`technician.latitude`, `technician.longitude`) from the worker profile and sends them as the origin of the transport request. The customer's selected home address is sent as the destination.
- **Flutter application → Component 4 (presentation).** The application calls both endpoints with standard HTTP POST requests, shows the repair range on the service selection screen, and shows the transport fee with its breakdown at checkout.

Component 3's trust score is not an input to either pricing model. The trust score influences which technicians are recommended (through Component 2), but a highly rated technician is not automatically priced higher. This keeps the transport fee tied to travel conditions and the repair range tied to the job.

In the current stage the Flutter application calls the two Render endpoints directly, which allowed each team member to develop and test their component independently. In the production architecture, the Node.js API gateway will sit between the application and Component 4. It will authenticate requests with JWT, attach booking metadata and combine the pricing responses with the booking record. This routing has not yet been implemented and is listed as future work.

Revenue rule. The whole transport fee is passed to the technician as reimbursement for fuel, vehicle wear and the congestion or weather risk. The platform takes no commission on the transport fee; commission applies only to the repair labour charge. This rule is implemented in the wider booking and payment flow rather than in Component 4, but it shaped the design: the fee is meant to compensate the technician, not to generate platform revenue.

5.4 Verification and software testing

Testing was carried out at two levels, following the separation of unit and system tests in ISO/IEC/IEEE 29119 [70]:

- **Unit and boundary tests** with pytest (`test_app.py`) checked the calculation logic and input validation locally. External API calls were simulated where needed, for example to reproduce an OpenWeatherMap timeout.
- **Endpoint tests** with a Postman collection sent requests to the live Render services and checked status codes and response fields.

Fourteen automated test cases were written: eight for the transport fee engine and six for the repair estimator. The conditions they cover are summarised in Table 5.2. All fourteen passed.

Table 5.2: Automated test coverage and results

Module	Conditions covered	Cases	Result
Transport fee engine	Out-of-bounds coordinates (e.g., latitude > 90); missing origin or destination keys in the JSON body; zero-distance trips with identical coordinates; negative values; simulated OpenWeatherMap timeout with fallback to the neutral multiplier	8	8 / 8 passed
Repair price estimator	Standard (known) categories; unseen categories handled by handle_unknown='ignore'; missing trade key; output always satisfies price_min ≤ price_max	6	6 / 6 passed
Total		**14**	**14 / 14 (100%)**

In addition, the documented transport example (5.4 km, heavy traffic, rain) was recalculated by hand (Section 4.4.3). The scenario values in Section 6.2 were computed directly from Equation (4.1) with the configured parameters. Model accuracy is reported separately in Section 6.1, because it is an evaluation of the learned model rather than a pass or fail software test.

The test suite has limits. It checks correctness and robustness of individual requests. It does not include load testing under many simultaneous users, long-running availability monitoring, or tests of the Google Maps failure path against the live API. These are listed as future work.

5.5 Deployment and operation

Both services are deployed as separate web services on Render. The transport fee engine runs as a stateless Linux service started by Gunicorn. The repair estimator is built from its Dockerfile and started by Uvicorn. Each service has its own public HTTPS URL, so one can be redeployed without affecting the other.

All secrets and tariff values are stored in Render's environment variable settings, following the Twelve-Factor principle of keeping configuration in the environment [61]. When CEYPETCO revises petrol prices, the operator updates `CURRENT_PETROL_PRICE` in the Render dashboard and the service restarts with the new value. No code change or new release of the mobile application is needed. Fuel prices in Sri Lanka are revised by announcement, at most a few times a month in 2026 [7]–[9], so manual updating is practical. It does, however, depend on someone remembering to make the update; automatic retrieval is future work.

Operational considerations for the current deployment are:

- **Cost of external APIs.** Each transport estimate makes one Distance Matrix call and one weather call. Google Maps Platform is billed per request beyond its free allowance, so the gateway should limit repeated requests from the same user.
- **Security.** The public endpoints do not yet require authentication or apply rate limits. This is acceptable for development but must be fixed before public launch through the Node.js gateway [67].
- **Idle behaviour.** Depending on the Render instance type, a service that has been idle may take longer to answer its first request. The latency in Section 6.2 refers to a running service.
- **Monitoring.** Render's service logs are used to check errors. Structured logging of external API failures and fallback use would help detect problems such as an expired API key.

6 RESULTS AND VALIDATION

This chapter reports the results of the evaluations described in Chapter 3. Section 6.1 gives the accuracy of the repair price models (RQ2). Section 6.2 gives the behaviour of the transport fee engine and the technical performance of both services (RQ1 and RQ3). Sections 6.3 and 6.4 report the status of user validation and integration, and Section 6.5 summarises the findings.

6.1 Research results

6.1.1 Accuracy of the repair price models

Table 6.1 gives the hold-out results of the multi-output decision tree and the random forest baseline on the 20% test split of the 630-row dataset. As explained in Section 3.5.1, the test rows are near-duplicates of training rows, so all of these figures are optimistic estimates of performance on genuinely new jobs.

Table 6.1: Hold-out results of the repair price models (random 80/20 split of the augmented dataset)

Model	Target	MAE (LKR)	RMSE (LKR)	R^2
Decision Tree (max_depth = 12)	price_min	1,712.04	2,250.31	0.8880
Decision Tree (max_depth = 12)	price_max	3,143.99	4,423.70	0.9245
Random Forest (100 trees)	price_min	1,758.31	2,227.66	0.8903
Random Forest (100 trees)	price_max	3,434.35	4,661.73	0.9161

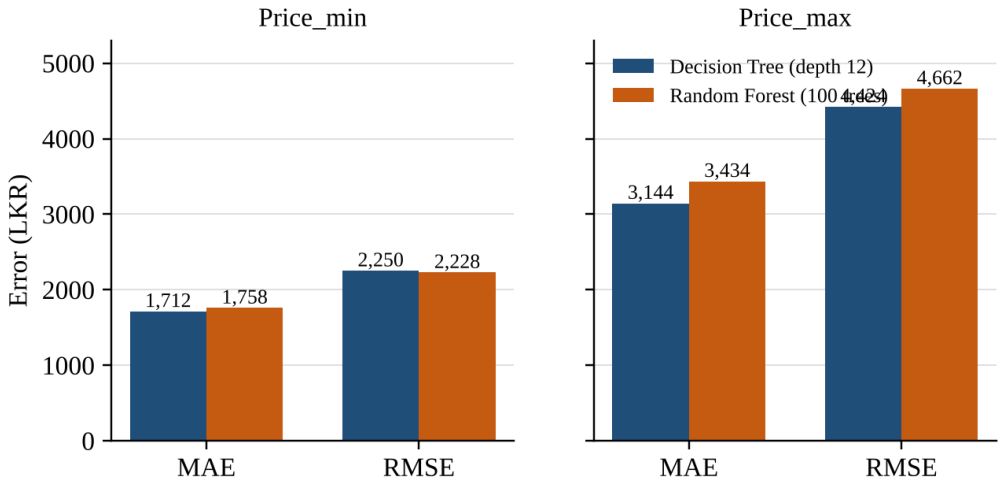


Figure 6.1: MAE and RMSE of the decision tree and random forest for each target

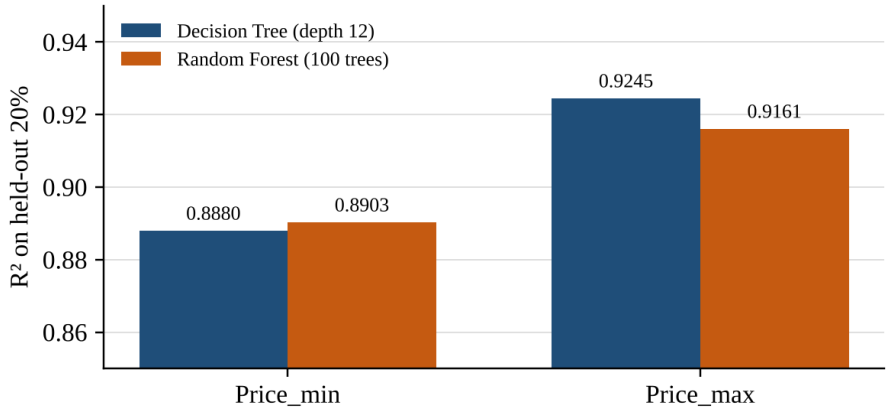


Figure 6.2: R^2 of the decision tree and random forest for each target (axis starts at 0.85)

The main observations are:

- **Both models explain most of the variance.** R^2 is between 0.888 and 0.925 for all four model–target pairs. The maximum price is predicted with a higher R^2 than the minimum price by both models.
- **The decision tree has lower average error.** Its MAE is 2.6% lower than the random forest's for price_min (LKR 1,712 vs 1,758) and 8.5% lower for price_max (LKR 3,144 vs 3,434).
- **The comparison on RMSE and R^2 is mixed.** For price_max the decision tree is better on both (RMSE 5.1% lower, R^2 0.9245 vs 0.9161). For price_min the random forest is slightly better (RMSE 1.0% lower, R^2 0.8903 vs 0.8880).
- **Errors are uneven.** The ratio of RMSE to MAE is 1.31 and 1.41 for the decision tree, and 1.27 and 1.36 for the random forest. A ratio clearly above 1 means that a minority of predictions have much larger errors than the typical one [53]. This fits the data construction in Section 3.2.2: expensive jobs carry most of the price variation.
- **Absolute errors are large compared with the rounding step.** An MAE of LKR 1,712 is more than three LKR 500 rounding steps. Because the price distribution of the dataset was not summarised in the evaluation output, the error cannot be stated as a percentage of the typical price. It is, however, small for high-value jobs such as display replacements or AC installation, and large for low-value jobs.

With only one random split, differences of a few percent between the two models cannot be shown to be statistically meaningful. The fair conclusion is that the two models have **comparable accuracy** on this dataset. The decision tree has lower MAE on both targets and is simpler, while the random forest is marginally better on two of the four squared-error measures. The working hypothesis in Section 1.6, that a single tree would match the forest, is supported. The stronger claim in the component specification, that the tree clearly outperforms the forest, is not supported by these results.

6.1.2 Validity of the predicted ranges

In every test case, including requests with categories not seen in training, the predicted minimum was less than or equal to the predicted maximum (Section 5.4). This follows from the multi-output leaf averaging explained in Section 4.4.4 and means that the application never has to display an inverted range.

6.2 Product and technical results

6.2.1 Behaviour of the transport fee engine

Table 6.2 and Figure 6.3 show the fee for typical trip lengths under four conditions, calculated with the configured parameters (B = LKR 200, r_d = LKR 80/km, F_c/F_b = 414/350). The regulated three-wheeler passenger tariff in the Western Province (LKR 110 for the first km, LKR 90 for each further km [25]) is included as a local reference.

Table 6.2: Transport fee (LKR) by distance and travel conditions

Distance	Normal, clear (×1.00)	Moderate traffic, clear (×1.15)	Heavy traffic, rain (×1.56)	Heavy traffic, thunderstorm (×1.755)	Three-wheeler tariff (reference)
2 km	389	448	607	683	200
5 km	673	774	1,050	1,181	470
10 km	1,146	1,318	1,788	2,012	920
15 km	1,619	1,862	2,526	2,842	1,370
20 km	2,093	2,406	3,264	3,672	1,820

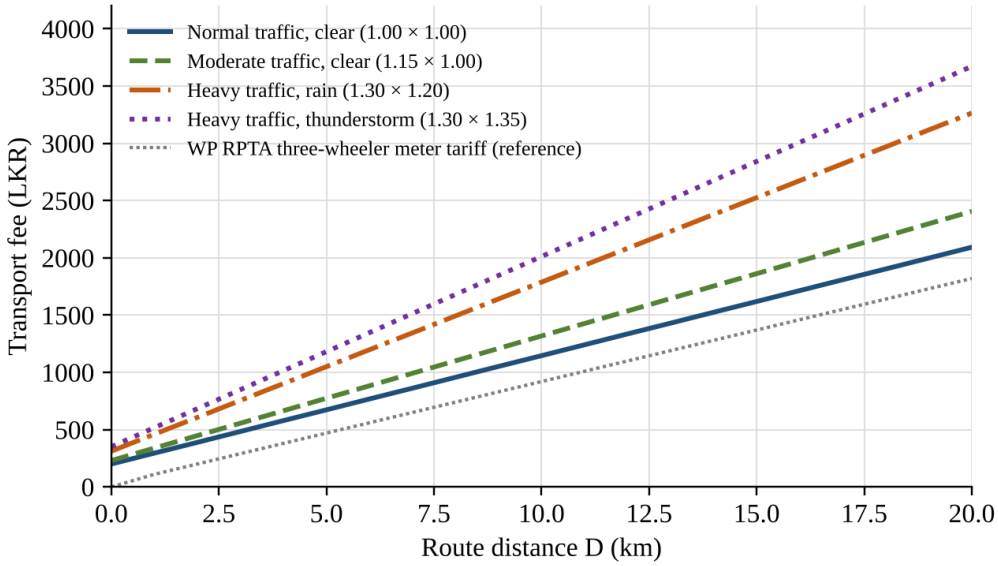


Figure 6.3: Transport fee against route distance for four travel conditions, with the three-wheeler tariff as a reference

The results show that the fee behaves as designed:

- In normal conditions the fee is above the three-wheeler passenger tariff, by 43% at 5 km and 25% at 10 km. The gap narrows with distance because the technician fee has a higher fixed part (the LKR 200 call-out charge) but a similar per-km rate after fuel indexation (LKR 94.6 vs LKR 90). A technician is therefore paid somewhat more than a passenger would pay for the same journey, which reflects the tools, time and return trip that the tariff does not cover.
- Each condition adds a visible premium. For a 10 km trip, heavy traffic in rain adds LKR 642 to the normal fee, and a thunderstorm with heavy traffic adds LKR 866.
- The fee rises steadily with every factor, and its upper limit for a given trip is known in advance (1.755 times the normal fee).

Figure 6.4 decomposes the API example from Section 4.4.3. Of the LKR 1,109 fee, LKR 200 is the base fare, LKR 511 the fuel-indexed distance cost, about LKR 213 the traffic premium and about LKR 185 the weather premium. This is the information returned to the application, so a customer can see that roughly a third of the fee is due to the heavy traffic and rain at that moment.

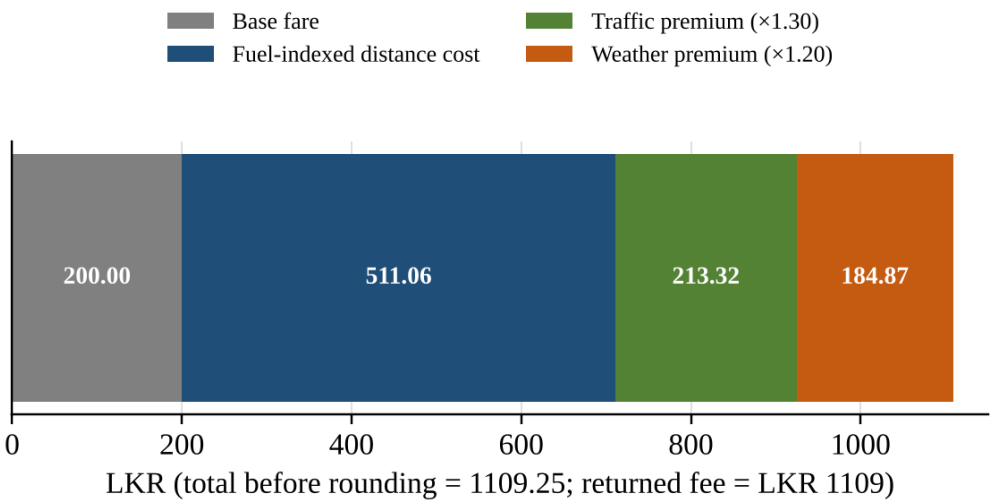


Figure 6.4: Composition of the LKR 1,109 example fee (5.4 km, heavy traffic, rain)

6.2.2 Sensitivity to fuel prices

Figure 6.5 and Table 6.3 show the fee for a 10 km trip in normal conditions under the petrol prices published during 2026. Between the lowest (LKR 398) and highest (LKR 434) prices in the year, the fee changed from LKR 1,110 to LKR 1,192, a difference of LKR 82 or 7.4%. Without fuel indexation, the fee would have stayed at LKR 1,000, so the technician would have absorbed the entire increase.

Table 6.3: Fee for a 10 km trip in normal conditions under 2026 petrol prices

Petrol 92 price (LKR/L)	Period	Fuel ratio	Fee (LKR)
350	Baseline F_b (configured reference)	1.000	1,000
398	Before 2 May 2026 [7]	1.137	1,110
410	From 2 May 2026 [7]	1.171	1,137
434	From 30 May 2026 [8]	1.240	1,192
399	Before 1 October 2026 [9]	1.140	1,112
414	From 1 October 2026 (current configuration) [9]	1.183	1,146

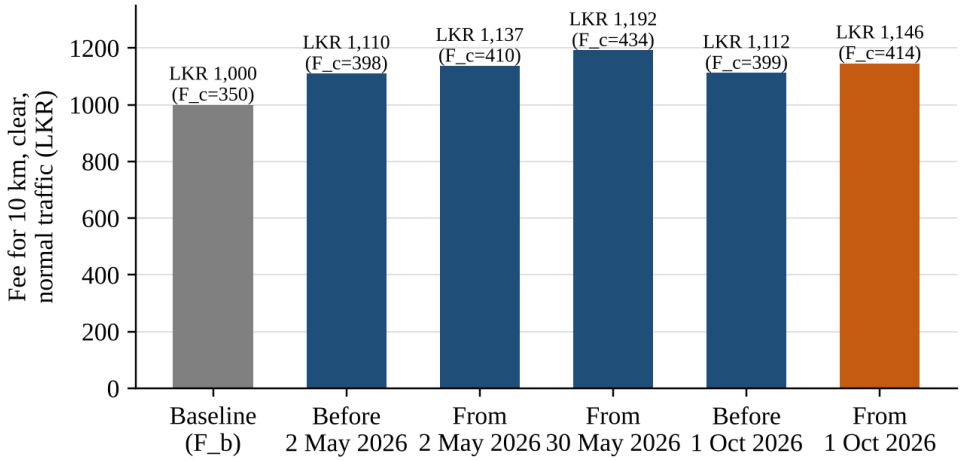


Figure 6.5: Fee for a 10 km trip in normal conditions under the petrol prices of 2026

6.2.3 Performance and reliability

The measured inference latency of the deployed decision tree pipeline on Render is shown in Figure 6.6. The mean was 11.10 ms, with a minimum of 6.54 ms and a maximum of 22.92 ms. The mean meets the 15 ms target (NFR1), but some requests took longer than 15 ms, so the target is met on average rather than for every request. These times cover processing inside the service. The time a user waits also includes the mobile network round trip to Render, which is usually much longer than the inference itself.

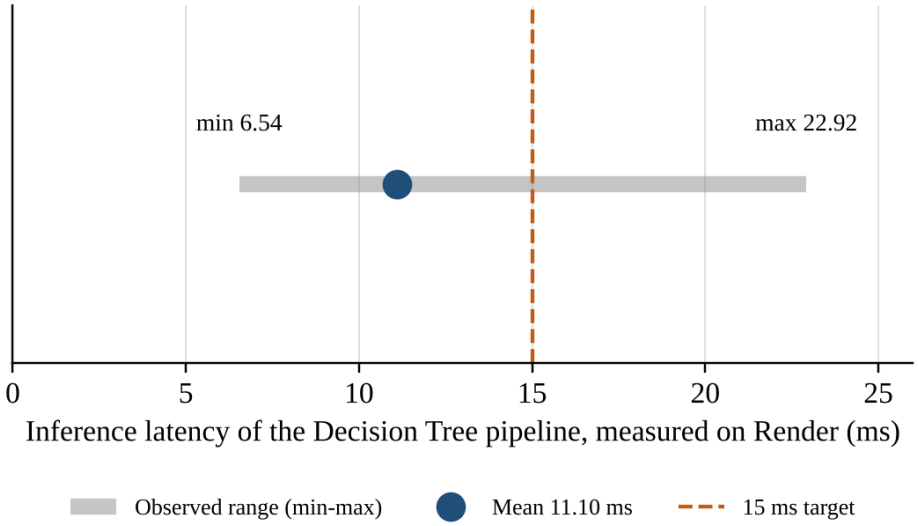


Figure 6.6: Measured inference latency of the repair price estimator

The transport engine’s response time is dominated by its two external API calls, and its own calculation is negligible. Its latency was not measured separately. All

fourteen automated tests passed (Section 5.4), including the weather timeout fallback and the handling of unseen repair categories, which shows that the services meet the robustness requirements NFR2 and FR9.

6.3 User or stakeholder validation

No validation with real customers or technicians has been carried out at the time of writing. The tradespeople who supplied prices took part only as data sources and did not evaluate the system. As a result, the outcomes in the conceptual framework that depend on people's responses have **not yet been tested**: technicians' willingness to accept jobs, customers' trust and perceived fairness (Figure 2.1).

A validation study is planned in two parts, as summarised in Table 6.4:

Table 6.4: Planned user and stakeholder validation

Part	Participants	Method	Measures
A. Technician acceptance	Independent technicians from at least three trades in the Western Province	Scenario-based survey: accept or decline hypothetical jobs with (i) a flat fee and (ii) the context-aware fee, across distances, traffic and weather	Acceptance rate; perceived fairness of compensation (Likert scale); comments on multipliers
B. Customer perception	Household customers who have hired a technician in the past year	Prototype walkthrough showing the repair range and itemised transport fee, followed by a questionnaire	Perceived price fairness [16]; trust in the estimate; usefulness of the breakdown; willingness to book
C. Price accuracy	Completed bookings during a pilot	Compare final agreed repair prices with the predicted ranges	Share of final prices inside the predicted range; MAE on real jobs

Part C is the most important for the repair model, because it would provide the real transaction data that the current synthetic augmentation cannot replace.

6.4 Integration results

Table 6.5 summarises the integration status of Component 4 with the rest of the solution.

Table 6.5: Integration status of Component 4

Integration point	Status	Notes
Flutter app → repair estimator	Implemented	Direct HTTPS POST to Render; range shown on service selection
Flutter app → transport fee engine	Implemented	Direct HTTPS POST at checkout; breakdown available for display

Integration point	Status	Notes
Component 1 worker types → repair dataset	Implemented	Same 39 labels; no mapping layer needed
Component 2 selected technician → transport origin	Implemented	technician.latitude / longitude used as origin; customer home as destination
Node.js API gateway routing (JWT, aggregation)	Planned	Required before public launch for authentication and rate limiting
Payment and commission rules	Outside component	Transport fee passed fully to the technician; commission only on labour

6.5 Summary of findings

Table 6.6: Summary of findings by research question

Research question	Finding	Evidence
RQ1: transparent context-aware transport fee	Supported. The fee is deterministic, itemised, monotonic in every factor, bounded (×1.755) and follows real fuel price changes (+7.4% across 2026 prices for 10 km).	Sections 4.4.1, 6.2.1, 6.2.2; documented example verified
RQ2: accuracy of multi-output decision tree	Partly supported. R^2 0.888 (min) and 0.925 (max); MAE LKR 1,712 and 3,144; comparable to random forest. Results are optimistic because of leakage and must be confirmed with a grouped split and real data.	Section 6.1; Section 3.5.1
RQ3: deployable low-latency microservices	Supported for the current stage. Both services are live on Render, 14/14 tests passed, mean inference latency 11.10 ms (max 22.92 ms). Authentication and load testing are pending.	Sections 5.4, 5.5, 6.2.3
Conceptual outcomes (acceptance, trust, fairness)	Not yet tested.	Section 6.3 (planned study)

7 DISCUSSION

7.1 Interpretation of findings

7.1.1 Transport fee engine

The transport fee results show that a simple formula can capture the main sources of travel cost while staying easy to explain. The design separates the fixed call-out cost, the distance cost and two condition premiums, and returns each one. This answers the main criticism of algorithmic pricing found in the literature, that workers and customers cannot see why a price is what it is [3], [19]. Because every premium is linked to a measurable cost driver (congestion ratio, rain, fuel price), the fee follows the principle that cost-justified price changes are perceived as fair [15], [16].

The fuel index is perhaps the most practical feature for Sri Lanka. In 2026 alone, petrol prices moved between LKR 398 and LKR 434 per litre in several steps [7]–[9]. A fixed per-km rate set when petrol cost LKR 398 would have under-paid a technician by about LKR 82 (around 7%) on a 10 km trip after the price rose to LKR 434 at the end of May. With indexation, a single configuration change keeps compensation in line with fuel costs.

The traffic and weather multipliers are stepped, not continuous. This makes them easy to communicate ("heavy traffic: ×1.30"), but it also means the fee can jump at the thresholds. A trip with a 39% delay pays the moderate rate while one with a 40% delay pays the heavy rate. The steps are 15 percentage points of the subtotal, which is noticeable but not extreme. The multiplier values themselves (1.15, 1.30, 1.20, 1.35) are design judgements informed by the literature on travel-time increases [4], [5]. They are not calibrated against technicians' actual acceptance behaviour, which is what Part A of the planned study would provide.

7.1.2 Repair price model

The repair models reach R^2 values close to 0.9, yet their mean absolute errors are in the thousands of rupees. Two features of the data and model explain this apparent contradiction.

First, the variance in the targets is dominated by expensive jobs. The ±8% noise added during augmentation spans several LKR 500 rounding steps for a high-value job but often rounds away completely for a cheap one (Section 3.2.2). Even a model that perfectly identified the job would therefore make errors of several thousand rupees on expensive jobs. These errors are noise that cannot be predicted, and they raise both MAE and RMSE. A high R^2 is still possible because the differences *between* job types, from a few thousand to tens of thousands of rupees, are much larger than the noise *within* a job type.

Second, the depth limit of 12 constrains how finely the tree can separate the jobs. With one-hot encoding, each split tests a single category value, such as one service. Separating 105 combinations one service at a time would need a much deeper tree than 12 levels in the worst case. The greedy algorithm splits first on the categories that reduce error most, which are usually the expensive ones. Cheaper, similar jobs may then share leaves and receive an averaged price. This also qualifies the claim that a single tree avoids "cross-category smoothing": a depth-limited tree can still average different services within a leaf. The difference from a forest is that the grouping is explicit and can be inspected. Inspecting the trained tree's leaves, and testing deeper trees, is a simple next step.

The leakage described in Section 3.5.1 means these results mainly show how well the models reproduce the prices of *known* job types. Neither model has been tested on job types it has never seen. For a truly new combination, `handle_unknown='ignore'` turns the unknown value into an all-zero vector. The tree then follows the "not equal" branches and returns the price of some default group of jobs. The test in Section 5.4 confirms that a valid range is returned in this case, but the range may not be meaningful. In practice this risk is limited because the application offers only the services and specifications in the dataset. It becomes important if new services are added without retraining.

The comparison between the decision tree and the random forest should be read with these points in mind. Both models were evaluated under the same conditions, so the comparison between them is fair. Their similar results suggest that, for this categorical problem, averaging many trees adds little accuracy. This agrees with the

argument that when accuracy is comparable, the interpretable model should be preferred [42].

7.2 Comparison with literature baselines and existing solutions

Dynamic pricing. Surge pricing models in ride-hailing set prices to balance supply and demand [2], [17]. Empirical studies show that drivers respond to these prices [10] and that removing them can cause the market to fail [18]. Component 4 takes a different position. Its premiums are triggered by cost conditions, not by demand, so it will not raise prices simply because many customers are booking at once. This gives up some of the market-balancing power of surge pricing in exchange for transparency and perceived fairness. That trade-off suits household repair, where customers are often in urgent situations such as a burst pipe or a failed refrigerator, and where demand-based surges could easily be seen as exploitation. The weather premium still keeps part of the supply effect observed by Brodeur and Nield [11], because it raises pay in exactly the conditions in which rides, and probably repair call-outs, become harder to fill.

Local fare structures. Nishantha proposed a cost-plus structure for three-wheeler fares to address unfair and inconsistent pricing [20], and the current regulated tariff follows a similar first-km-plus-per-km pattern [25]. Component 4 extends that structure in three ways: a call-out charge that reflects a technician's journey rather than a passenger trip, automatic fuel indexation, and real-time condition multipliers. Its normal-condition fee is 25–43% above the passenger tariff for 5–10 km trips, which is a plausible premium for a skilled worker's travel. Whether technicians and customers agree is a question for the user study.

Repair cost estimation. The R^2 values of 0.89–0.92 cannot be compared directly with those of studies that use real transaction data. Dzięcioł and Wrzesiński reported an R^2 of only 0.452 for real residential building repair costs [22], and Elmousalami noted that data quality limits accuracy in construction cost estimation [21]. Their problems are harder because real costs vary for reasons that are not recorded. The present dataset, by contrast, was built from 105 surveyed ranges with controlled noise. The higher R^2 here reflects the cleaner, synthetic structure of the data and the

leakage, not a better method. The fair comparison is qualitative: this work shows that a tree-based model can learn a structured price catalogue and serve it at low latency. Accuracy on real Sri Lankan transactions is still unknown.

Existing products. Compared with the systems in Table 2.2, Component 4 combines two features that were not found together: a real-time, condition-aware travel fee and a data-driven repair price range shown before the visit. Fixed-menu home service apps also show prices in advance, but these are set by the company. Quotation marketplaces reveal prices only after workers respond. The estimator's ranges are learned from local market rates and can be retrained as data accumulates.

7.3 Research contribution

The research contributions of this work are:

1. **A context-aware cost-plus model for technician travel fees**, combining live road distance, a travel-time-index-style congestion ratio, destination weather and national fuel price indexation in a single transparent, bounded formula. The model's properties (floor, linearity, monotonicity, bounded premium) are stated and verified.
2. **A structured repair price catalogue for Sri Lankan household services**, with 105 base combinations across 39 trades, 95 services and 91 specifications, aligned with the worker categories of an intent recognition model. Such data was not previously available in structured form.
3. **An empirical comparison of a multi-output decision tree and a random forest** for categorical price-range estimation, showing comparable accuracy and supporting the choice of the interpretable model.
4. **A documented case of evaluation leakage caused by augmentation before splitting**, with a quantitative estimate of its extent (Section 3.5.1). This is a practical lesson for student and practitioner projects that augment small datasets [50], [54].

7.4 Product contribution

The product contributions are two deployed microservices that can be used by the marketplace now and adapted later:

- A **transport fee API** that returns an exact, itemised fee in one call, with server-side secrets, configurable tariffs and fault-tolerant weather handling.
- A **repair estimate API** that returns a minimum–maximum range in LKR fast enough to update as the user changes selections.
- A **booking flow** in which the customer sees an indicative repair range before choosing a technician and an exact travel fee before confirming. This reduces the uncertainty that usually surrounds informal repair bookings.
- A **revenue rule** that passes the whole transport fee to the technician, which makes the fee a credible incentive rather than a platform charge.

7.5 Limitations and threats to validity

The limitations are grouped by type of validity in Table 7.1.

Table 7.1: Limitations and threats to validity

Type	Threat	Effect	Mitigation / future action
Internal	Augmentation was done before the random train/test split, so near-duplicates of test rows are in the training set	MAE, RMSE and R^2 are optimistic; performance on new job types is unknown	Grouped split by base combination; report both results
Internal	Single random split; no cross-validation or confidence intervals	Small differences between models may be due to chance	Repeated grouped cross-validation [51]
Internal	Random seed and price distribution of the dataset not reported with the results	Exact split and relative error cannot be reproduced from the thesis alone	Record seed and descriptive statistics in the repository
Construct	Targets are surveyed typical ranges with synthetic noise, not real transaction prices	Model learns a price catalogue, not market behaviour	Collect real booking prices (Section 6.3, Part C)
Construct	Multiplier values and traffic thresholds are design judgements	Fee may over- or under-compensate in some conditions	Calibrate with technician acceptance data
External	Survey size and coverage limited; prices mainly urban and 2026	Ranges may not hold in other districts or later years	Wider survey; periodic retraining; regional features
External	No user study	Effects on acceptance, trust and fairness are untested	Planned study (Section 6.3)
Conclusion	Latency measured on a running service; network time excluded; max 22.92 ms exceeds target	Real user waiting time is longer and variable	End-to-end measurement from devices; load testing
Operational	Fuel price updated manually; no Google Maps fallback; no authentication on public endpoints	Stale fees, failed estimates during outages, API misuse	Automated price retrieval; haversine fallback [56]; gateway with JWT and rate limiting

The most serious limitation is the evaluation leakage, because it affects the main quantitative result of the thesis. It does not make the model useless: the application

only asks for job types that exist in the dataset, which is the situation the hold-out test most closely resembles. It does mean that the reported accuracy should not be quoted as evidence that the model can price new kinds of jobs, and that the comparison with real-data studies in Section 7.2 must remain qualitative.

7.6 Implications

For the platform. The two services can be used as they are for a pilot, provided the repair range is presented as an indication and not a quotation, and the gateway adds authentication before public launch. The pilot should be used to collect the real prices and acceptance decisions needed to recalibrate both modules.

For technicians. A travel fee that rises with distance, congestion, weather and fuel removes the main financial reason to refuse distant or difficult jobs. Passing 100% of the fee to the technician makes this incentive credible.

For customers. Seeing a repair range before the visit and an itemised travel fee before confirming reduces the information gap typical of credence services [13]. It gives the customer a reference point for judging the technician's quotation [31].

For policy. The approach shows how a published tariff structure, such as the three-wheeler meter tariff, could be extended with transparent indexation to fuel prices. This might reduce the need for frequent manual tariff revisions.

For researchers and students. The leakage identified here is easy to introduce when small datasets are augmented. Splitting before augmentation, or grouping by source record, should be standard practice [50].

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

Component 4 is not sold as a separate product to households; it creates value inside the marketplace by making bookings easier to agree. Its value proposition differs for each user group, as shown in Table 8.1.

Table 8.1: Value proposition by user group

User group	Pain point	Value delivered by Component 4
Household customers	Do not know what a repair should cost; fear over-charging; travel charges are negotiated ad hoc	Indicative repair range before choosing a technician; exact, itemised travel fee before confirming; no demand-based surge
Technicians and workshops	Travel to distant jobs, in traffic or rain, is unpaid or underpaid; fuel price increases reduce income	Travel fee that grows with distance, congestion, weather and fuel price; 100% of the fee passed to the technician; fewer price disputes on site
Platform operator	Unfilled requests at busy times or in bad weather; disputes damage trust	Higher job acceptance in difficult conditions; consistent, auditable prices; configurable tariffs without app releases
Other service businesses (future)	Need travel or job price estimates for their own field workers	Pricing APIs that can be licensed or adapted

The primary target market for the first release is urban and suburban households in the Western Province, where traffic congestion is most severe and smartphone use is high, together with the independent technicians who serve them. The 39 trades covered by the dataset include the most frequent household needs, such as air-conditioning, plumbing, electrical work, appliance repair, painting and carpentry.

8.2 Market and competitive position

Services employed about 4.28 million people in Sri Lanka in the fourth quarter of 2025, 50.6% of all employment [1]. Household repair work is a small but widespread part of this sector and is mostly informal. Globally, digital labour platforms have grown rapidly, and location-based platforms that match local workers with customers form a major part of this growth [72], [73]. In Sri Lanka, ride-hailing platforms have made app-based, metered pricing familiar for transport, which lowers the barrier to accepting algorithmic travel fees for technicians.

Table 8.2 positions the marketplace, with Component 4, against the alternatives a Sri Lankan household currently uses.

Table 8.2: Competitive position of the pricing approach

Alternative	Price transparency before visit	Travel compensation	Main weakness
Word of mouth / phone call	None	Negotiated	No reference price; strong information asymmetry
Classified listing sites	Occasionally an advertised starting price	Negotiated	Prices not comparable; no verification
Fixed-menu home service apps	Fixed company price	Fixed visiting charge	Prices not linked to local market data or conditions
Quotation marketplaces	After quotes arrive	Included in quote	Customer must wait for and compare quotes
This marketplace (with Component 4)	Data-driven range at service selection	Context-aware, itemised, fuel-indexed	Needs real-price calibration and user validation

The competitive advantage of the pricing engine lies less in any single feature than in combining three elements: local price data aligned with the platform's own worker categories, real-time condition pricing for travel, and complete transparency of the calculation. Two-sided platforms gain value as more participants join each side [74], and fair, predictable prices help attract both technicians and customers. The data advantage also grows over time, because every completed booking can add a real price to the dataset, which competitors without the same booking flow cannot easily copy.

8.3 Business or sustainability model

The business model of the pricing engine is summarised in the business model canvas of Table 8.3 [75]. Its main role is to support the marketplace's revenue rather than to earn revenue directly. Because platform commission applies only to repair labour, a better repair estimate and higher job acceptance increase the number and value of completed jobs on which commission is earned.

Table 8.3: Business model canvas for the pricing engine

Building block	Content
Customer segments	Household customers; independent technicians and small workshops; (later) other field-service businesses
Value propositions	Transparent repair price range; fair, condition-aware travel fee; fewer disputes; higher acceptance (Section 8.1)
Channels	Flutter mobile application; later, B2B pricing APIs

Building block	Content
Customer relationships	Self-service in app; price breakdown builds trust; feedback on estimates collected after jobs
Revenue streams	Indirect: platform commission on repair labour enabled by more completed bookings. Potential direct: licensing the pricing APIs; premium market-rate insights for workshops. No commission on transport fees
Key resources	Repair price dataset; fee model and parameters; trained model; cloud services
Key activities	Updating fuel prices; collecting real job prices; retraining; monitoring API usage
Key partners	Google Maps Platform; OpenWeatherMap; Render; technician associations and workshops (data)
Cost structure	Per-request mapping and weather API charges beyond free allowances; cloud hosting; data collection and maintenance effort

The main variable cost is the external API usage: each transport estimate uses one routing call and one weather call. Two design measures keep this cost proportional to real bookings. The fee is requested only after a technician is selected, not while browsing, and the repair estimate uses no external API at all. Further savings are possible by caching weather per area for a few minutes, because weather does not change between requests seconds apart.

Sustainability of the model depends on keeping the data current. Repair prices will drift with inflation and with the cost of parts. The planned approach is to compare each predicted range with the final agreed price (Section 6.3, Part C), monitor the share of prices falling outside the range, and retrain when it rises. Fuel indexation already keeps the transport fee current between reviews of the base parameters.

8.4 Scalability intellectual property and compliance

Scalability. Both services are stateless. Each request carries everything needed to compute the answer, so the services can be scaled horizontally by running more instances behind Render's load balancing, without shared state. The repair estimator keeps the model in memory and needs only a few milliseconds of computation per request (Section 6.2.3). The transport engine's throughput is limited mainly by the external APIs' quotas, which can be raised by plan. The microservice separation means that a surge in estimate requests does not affect fee calculation, and the reverse [59]. Extending to new regions requires only new price data and, if necessary, regional tariff parameters.

Intellectual property. The original assets are the fee model design and parameters, the repair price dataset and the trained model, and the service code. These are protected as copyright and as confidential know-how of the project. The dataset is the most valuable asset, because it is hard to reproduce without field work. The software uses open-source components under permissive licences (Flask and scikit-learn under BSD licences, FastAPI under the MIT licence), which allow commercial use with attribution. Use of Google Maps and OpenWeatherMap data must follow those providers' terms of service, including restrictions on storing their data.

Compliance. Location coordinates of technicians and customers are personal data under the Personal Data Protection Act, No. 9 of 2022 [66]. The transport service processes them only to compute the fee, does not store them, and shares them only with the routing and weather providers. A production deployment must document this processing in the platform's privacy notice, establish a lawful basis (normally performance of the booking contract) and cover cross-border transfer to the API providers. Prices shown to consumers should be clear and not misleading, in line with the Consumer Affairs Authority Act, No. 9 of 2003 [71]. For this reason the repair range is labelled as an estimate, and the transport fee is shown with its breakdown before the customer confirms.

8.5 Contribution to the Sustainable Development Goals

The component contributes to several of the United Nations Sustainable Development Goals (SDGs) [76], as shown in Table 8.4.

Table 8.4: Contribution to the Sustainable Development Goals

SDG	Relevant target area	Contribution of Component 4
SDG 8: Decent work and economic growth	Productive employment and decent work; protection of labour rights	Pays technicians for real travel costs and risks; passes 100% of the travel fee to the worker; protects income against fuel price increases
SDG 9: Industry, innovation and infrastructure	Access of small enterprises to digital services	Gives independent technicians and small workshops access to data-driven pricing tools they could not build themselves
SDG 10: Reduced inequalities	Equal opportunity; reducing inequalities of outcome	Same fee for the same trip regardless of who the customer is; no demand-based surge; transparent prices reduce exploitation of less informed customers
SDG 11: Sustainable cities and communities	Safe, inclusive urban services	Weather premium encourages service availability during storms while recognising rider safety; supports maintenance of urban housing

SDG	Relevant target area	Contribution of Component 4
SDG 12: Responsible consumption and production	Consumer information	Price ranges help consumers make informed repair decisions, which can favour repair over replacement

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis addressed two pricing problems that discourage the use of digital platforms for household maintenance in Sri Lanka. Technicians are not fairly compensated for travel in traffic, rain and periods of rising fuel prices. Customers have no reliable reference for what a repair should cost. Component 4 of the group's AI-powered service marketplace responds with a pricing engine made of two independent microservices.

The Context-Aware Dynamic Transport Fee Engine prices the technician's journey with a transparent cost-plus formula: a LKR 200 base fare plus a LKR 80 per km distance cost indexed to the CEYPETCO petrol price, multiplied by a traffic multiplier of 1.00, 1.15 or 1.30 and a weather multiplier of 1.00, 1.20 or 1.35. The fee is deterministic, itemised, monotonic in each factor and bounded at 1.755 times the normal fee, and it follows real fuel price changes. Across the petrol prices published in 2026, the fee for a 10 km trip adjusted by about 7% without any code change.

The Machine Learning Repair Price Range Estimator predicts a minimum and maximum price for a worker type, service and specification with a multi-output decision tree. It was trained on 630 rows derived from 105 surveyed Sri Lankan price combinations covering 39 trades. On a random hold-out split it reached R^2 of 0.888 and 0.925 and MAE of LKR 1,712 and 3,144 for the minimum and maximum prices. This was comparable to a 100-tree random forest, with lower MAE on both targets. These results are optimistic because augmented copies of the same job were present in both training and test sets, and this limitation is reported throughout the thesis.

Both services are deployed on Render, pass all 14 automated tests, and integrate with the Flutter application and with Components 1 and 2. The estimator's mean measured inference latency is 11.10 ms. The work shows that transparent, locally grounded pricing for household services is technically feasible and can be delivered as low-latency services. Whether it improves technician acceptance, customer trust and perceived fairness must now be tested with real users.

9.2 Achievement of objectives

Table 9.1: Achievement of the specific objectives

Objective	Status	Evidence
O1: Review literature and justify design	Achieved	Chapter 2; conceptual framework (Figure 2.1)
O2: Design context-aware cost-plus fee model	Achieved	Equation (4.1), Table 4.4, Table 4.5, Table 4.6; properties in Section 4.4.1
O3: Build dataset and train multi-output model	Achieved, with limitations	630-row dataset; decision tree pipeline; survey size to be documented; synthetic augmentation
O4: Compare decision tree with random forest	Achieved	Table 6.1; comparable accuracy, leakage disclosed
O5: Deploy robust cloud microservices	Largely achieved	Live Render services; 14/14 tests; mean latency 11.10 ms; authentication and load testing pending
O6: Integrate and assess commercial viability	Partly achieved	Direct Flutter integration and alignment with Components 1 and 2; gateway routing planned; Chapter 8

9.3 Individual research and product contributions

The author's individual research contributions are the context-aware cost-plus model for technician travel fees, the structured repair price catalogue for Sri Lankan household services aligned with Component 1's worker categories, the empirical comparison of decision tree and random forest models for categorical price-range estimation, and the documented analysis of augmentation-induced leakage. The product contributions are the two deployed REST microservices, their automated tests and their integration into the booking flow, where customers see a repair range before choosing a technician and an exact, itemised travel fee before confirming.

9.4 Future directions

The following work is recommended, in order of priority:

1. **Re-evaluate the repair model without leakage.** Split the 105 base combinations into training and test groups before augmentation, or use grouped cross-validation, and report both the original and the corrected results. Record the random seed and descriptive statistics of the prices so that relative errors can be reported.

2. **Collect real prices and validate with users.** Run the planned study (Section 6.3): technician acceptance scenarios, customer perception of fairness and trust, and comparison of predicted ranges with final agreed prices during a pilot.
3. **Calibrate the transport parameters.** Use acceptance data to tune the base fare, distance rate, thresholds and multipliers. Consider a smooth traffic multiplier to remove jumps at the thresholds.
4. **Improve the repair model.** Inspect the trained tree's leaves for mixed services, test deeper trees, add region and season features, and consider quantile methods [47] once individual transaction prices are available. Publish a short model card describing intended use and limitations [77].
5. **Automate fuel price updates.** Retrieve CEYPETCO price revisions automatically, with a manual override and an alert when the stored price is out of date.
6. **Add a routing fallback.** When the Distance Matrix API fails, estimate distance as the haversine distance multiplied by a circuity factor of about 1.2–1.4 [55]–[57], and mark the fee as estimated.
7. **Harden for production.** Route calls through the Node.js gateway with JWT authentication and rate limiting [67], add load tests and end-to-end latency measurement from mobile devices, cache weather by area, and add structured logging of fallbacks.
8. **Extend the fee model.** Consider return-trip costs for remote locations, waiting time, and parking or toll charges where relevant.

REFERENCES

[1] Department of Census and Statistics, "Sri Lanka Labour Force Survey Quarterly Report – 2025 Fourth Quarter," Colombo, Sri Lanka, 2026.

[2] G. P. Cachon, K. M. Daniels, and R. Lobel, "The role of surge pricing on a service platform with self-scheduling capacity," *Manufacturing & Service Operations Management*, vol. 19, no. 3, pp. 368–384, 2017.

[3] A. Rosenblat and L. Stark, "Algorithmic labor and information asymmetries: A case study of Uber's drivers," *International Journal of Communication*, vol. 10, pp. 3758–3784, 2016.

[4] I. Tsapakis, T. Cheng, and A. Bolbol, "Impact of weather conditions on macroscopic urban travel times," *Journal of Transport Geography*, vol. 28, pp. 204–211, 2013.

[5] M. J. Koetse and P. Rietveld, "The impact of climate change and weather on transport: An overview of empirical findings," *Transportation Research Part D: Transport and Environment*, vol. 14, no. 3, pp. 205–221, 2009.

[6] Central Bank of Sri Lanka, *Annual Report 2022*. Colombo, Sri Lanka: CBSL, 2023.

[7] "Fuel prices increased," *Ada Derana*, May 2, 2026. [Online]. Available: https://www.adaderana.lk/news/122005/fuel-prices-increased

[8] "CPC raises prices of diesel, petrol, kerosene," *News 1st*, May 30, 2026. [Online]. Available: https://english.newsfirst.lk/2026/05/30/sri-lanka-s-cpc-raises-prices-of-diesel-petrol-kerosene

[9] "CPC revises fuel prices from midnight today (30)," *News 1st*, Sep. 30, 2026. [Online]. Available: https://english.newsfirst.lk/2026/09/30/cpc-revises-fuel-prices-from-midnight-today-30

[10] M. K. Chen and M. Sheldon, "Dynamic pricing in a labor market: Surge pricing and flexible work on the Uber platform," in *Proc. ACM Conf. Economics and Computation (EC '16)*, Maastricht, The Netherlands, 2016, p. 455.

[11] A. Brodeur and K. Nield, "An empirical analysis of taxi, Lyft and Uber rides: Evidence from weather shocks in NYC," *Journal of Economic Behavior & Organization*, vol. 152, pp. 1–16, 2018.

[12] M. R. Darby and E. Karni, "Free competition and the optimal amount of fraud," *Journal of Law and Economics*, vol. 16, no. 1, pp. 67–88, 1973.

[13] U. Dulleck and R. Kerschbamer, "On doctors, mechanics, and computer specialists: The economics of credence goods," *Journal of Economic Literature*, vol. 44, no. 1, pp. 5–42, 2006.

[14] H. S. Schneider, "Agency problems and reputation in expert services: Evidence from auto repair," *Journal of Industrial Economics*, vol. 60, no. 3, pp. 406–433, 2012.

[15] D. Kahneman, J. L. Knetsch, and R. Thaler, "Fairness as a constraint on profit seeking: Entitlements in the market," *American Economic Review*, vol. 76, no. 4, pp. 728–741, 1986.

[16] L. Xia, K. B. Monroe, and J. L. Cox, "The price is unfair! A conceptual framework of price fairness perceptions," *Journal of Marketing*, vol. 68, no. 4, pp. 1–15, 2004.

[17] J. C. Castillo, D. Knoepfle, and G. Weyl, "Surge pricing solves the wild goose chase," in *Proc. ACM Conf. Economics and Computation (EC '17)*, Cambridge, MA, USA, 2017, pp. 241–242.

[18] J. Hall, C. Kendrick, and C. Nosko, "The effects of Uber's surge pricing: A case study," Univ. of Chicago Booth School of Business, Working Paper, 2015.

[19] U. M. Dholakia, "Everyone hates Uber's surge pricing – here's how to fix it," *Harvard Business Review*, Dec. 2015.

[20] K. M. Nishantha, "Development of a fare structure for the three wheelers," M.Eng. thesis, Dept. of Civil Engineering, University of Moratuwa, Sri Lanka, 2017.

[21] H. H. Elmousalami, "Artificial intelligence and parametric construction cost estimate modeling: State-of-the-art review," *Journal of Construction Engineering and Management*, vol. 146, no. 1, Art. no. 03119008, 2020.

[22] J. Dzięcioł and G. Wrzesiński, "Estimation of residential building repair costs using selected machine learning algorithms," *Buildings*, vol. 16, no. 16, Art. no. 3304, 2026.

[23] B. Park and J. K. Bae, "Using machine learning algorithms for housing price prediction: The case of Fairfax County, Virginia housing data," *Expert Systems with Applications*, vol. 42, no. 6, pp. 2928–2934, 2015.

[24] S. Pudaruth, "Predicting the price of used cars using machine learning techniques," *International Journal of Information & Computation Technology*, vol. 4, no. 7, pp. 753–764, 2014.

[25] Z. Farzan, "First kilometer fare for three-wheelers raised to Rs. 110/-," *News 1st*, Mar. 23, 2026. [Online]. Available: https://newsfirst.lk/2026/03/23/first-kilometer-fare-for-three-wheelers-raised-to-rs-110

[26] Google, "Distance Matrix API: Overview and request parameters (duration_in_traffic)," Google Maps Platform Documentation. [Online]. Available: https://developers.google.com/maps/documentation/distance-matrix/overview (accessed Oct. 9, 2026).

[27] OpenWeather Ltd., "Current weather data API and weather condition codes," OpenWeatherMap Documentation. [Online]. Available: https://openweathermap.org/current (accessed Oct. 9, 2026).

[28] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com (accessed Oct. 9, 2026).

[29] Pallets Projects, "Flask documentation (3.x)." [Online]. Available: https://flask.palletsprojects.com (accessed Oct. 9, 2026).

[30] G. A. Akerlof, "The market for "lemons": Quality uncertainty and the market mechanism," *Quarterly Journal of Economics*, vol. 84, no. 3, pp. 488–500, 1970.

[31] A. Tversky and D. Kahneman, "Judgment under uncertainty: Heuristics and biases," *Science*, vol. 185, no. 4157, pp. 1124–1131, 1974.

[32] C. Soh, M. L. Markus, and K. H. Goh, "Electronic marketplaces and price transparency: Strategy, information technology, and success," *MIS Quarterly*, vol. 30, no. 3, pp. 705–723, 2006.

[33] A. Hinterhuber, "Customer value-based pricing strategies: Why companies resist," *Journal of Business Strategy*, vol. 29, no. 4, pp. 41–50, 2008.

[34] A. V. den Boer, "Dynamic pricing and learning: Historical origins, current research, and new directions," *Surveys in Operations Research and Management Science*, vol. 20, no. 1, pp. 1–18, 2015.

[35] D. Schrank, B. Eisele, and T. Lomax, "2019 Urban Mobility Report," Texas A&M Transportation Institute, College Station, TX, USA, 2019.

[36] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

[37] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[38] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[39] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[40] H. Borchani, G. Varando, C. Bielza, and P. Larrañaga, "A survey on multi-output regression," *WIREs Data Mining and Knowledge Discovery*, vol. 5, no. 5, pp. 216–233, 2015.

[41] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[42] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[43] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv:1702.08608, 2017.

[44] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022.

[45] P. Cerda and G. Varoquaux, "Encoding high-cardinality string categorical variables," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 3, pp. 1164–1176, 2022.

[46] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.

[47] N. Meinshausen, "Quantile regression forests," *Journal of Machine Learning Research*, vol. 7, pp. 983–999, 2006.

[48] A. J. Wood, M. Graham, V. Lehdonvirta, and I. Hjorth, "Good gig, bad gig: Autonomy and algorithmic control in the global gig economy," *Work, Employment and Society*, vol. 33, no. 1, pp. 56–75, 2019.

[49] A. Hannák, C. Wagner, D. Garcia, A. Mislove, M. Strohmaier, and C. Wilson, "Bias in online freelance marketplaces: Evidence from TaskRabbit and Fiverr," in *Proc. ACM Conf. Computer Supported Cooperative Work and Social Computing (CSCW '17)*, Portland, OR, USA, 2017, pp. 1914–1933.

[50] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023.

[51] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Montreal, Canada, 1995, pp. 1137–1143.

[52] C. J. Willmott and K. Matsuura, "Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance," *Climate Research*, vol. 30, no. 1, pp. 79–82, 2005.

[53] T. Chai and R. R. Draxler, "Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature," *Geoscientific Model Development*, vol. 7, no. 3, pp. 1247–1250, 2014.

[54] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, Art. no. 15, 2012.

[55] R. H. Ballou, H. Rahardja, and N. Sakai, "Selected country circuity factors for road travel distance estimation," *Transportation Research Part A: Policy and Practice*, vol. 36, no. 9, pp. 843–848, 2002.

[56] F. P. Boscoe, K. A. Henry, and M. S. Zdeb, "A nationwide comparison of driving distance versus straight-line distance to hospitals," *The Professional Geographer*, vol. 64, no. 2, pp. 188–196, 2012.

[57] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[58] N. Dragoni *et al.*, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216.

[59] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[60] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. of Information and Computer Science, University of California, Irvine, CA, USA, 2000.

[61] A. Wiggins, "The Twelve-Factor App," 2011. [Online]. Available: https://12factor.net

[62] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

[63] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[64] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[65] C. M. Bishop, "Training with noise is equivalent to Tikhonov regularization," *Neural Computation*, vol. 7, no. 1, pp. 108–116, 1995.

[66] Parliament of the Democratic Socialist Republic of Sri Lanka, *Personal Data Protection Act, No. 9 of 2022*. Colombo, Sri Lanka: Government Publications Bureau, 2022.

[67] OWASP Foundation, "OWASP API Security Top 10 – 2023." [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (accessed Oct. 9, 2026).

[68] Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model, ISO/IEC 25010:2023, 2023.

[69] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, Art. no. 2, 2014.

[70] Software and Systems Engineering – Software Testing – Part 1: General Concepts, ISO/IEC/IEEE 29119-1:2022, 2022.

[71] Parliament of the Democratic Socialist Republic of Sri Lanka, *Consumer Affairs Authority Act, No. 9 of 2003*. Colombo, Sri Lanka: Government Publications Bureau, 2003.

[72] International Labour Organization, *World Employment and Social Outlook 2021: The Role of Digital Labour Platforms in Transforming the World of Work*. Geneva, Switzerland: ILO, 2021.

[73] M. Kenney and J. Zysman, "The rise of the platform economy," *Issues in Science and Technology*, vol. 32, no. 3, pp. 61–69, 2016.

[74] G. G. Parker, M. W. Van Alstyne, and S. P. Choudary, *Platform Revolution: How Networked Markets Are Transforming the Economy and How to Make Them Work for You*. New York, NY, USA: W. W. Norton, 2016.

[75] A. Osterwalder and Y. Pigneur, *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Hoboken, NJ, USA: Wiley, 2010.

[76] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

[77] M. Mitchell *et al.*, "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency (FAT* '19)*, Atlanta, GA, USA, 2019, pp. 220–229.

GLOSSARY

Cost-plus pricing – Setting a price as an estimate of cost plus a margin or premium.

Credence good – A good or service whose quality or necessity the buyer cannot fully judge even after buying it, such as many repairs.

Data augmentation – Creating additional training rows from existing ones, here by adding small random noise to prices.

Data leakage – Information in the training data that would not be available when the model is used, which makes evaluation results look better than they are.

Fuel index ratio – Current petrol price divided by a baseline price; used to scale the per-kilometre rate.

Hold-out set – Part of the data kept aside during training and used only to evaluate the model.

Leaf (decision tree) – An end node of a decision tree; its prediction is the mean of the training targets that reach it.

Microservice – A small, independently deployable service that performs one business function and communicates through a network interface.

Multi-output regression – Predicting several numeric targets with one model, here the minimum and maximum price.

One-hot encoding – Representing a categorical value as a vector of zeros with a single one in the position of that category.

Surge pricing – Raising prices when demand exceeds supply, as used in ride-hailing.

Traffic ratio / Travel Time Index – Travel time in current traffic divided by normal travel time.

APPENDICES

Appendix A Extended results

Table A.1 lists the transport fee for distances from 1 to 20 km under all combinations of the traffic and weather multipliers that differ from the normal case, using the configured parameters (B = LKR 200, r_d = LKR 80/km, F_c = LKR 414, F_b = LKR 350). Values are rounded to the nearest rupee, as returned by the service.

Table A.1: Transport fee (LKR) by distance and multiplier combination

Distance	1.00 × 1.00	1.15 × 1.00	1.30 × 1.00	1.00 × 1.20	1.00 × 1.35	1.30 × 1.20	1.30 × 1.35
1 km	295	339	383	354	398	460	517
2 km	389	448	506	467	525	607	683
3 km	484	556	629	581	653	755	849
4 km	579	665	752	694	781	902	1,015
5 km	673	774	875	808	909	1,050	1,181
6 km	768	883	998	921	1,036	1,198	1,347
7 km	862	992	1,121	1,035	1,164	1,345	1,514
8 km	957	1,101	1,244	1,148	1,292	1,493	1,680
9 km	1,052	1,209	1,367	1,262	1,420	1,641	1,846
10 km	1,146	1,318	1,490	1,376	1,547	1,788	2,012
11 km	1,241	1,427	1,613	1,489	1,675	1,936	2,178
12 km	1,336	1,536	1,736	1,603	1,803	2,083	2,344
13 km	1,430	1,645	1,859	1,716	1,931	2,231	2,510
14 km	1,525	1,754	1,982	1,830	2,058	2,379	2,676
15 km	1,619	1,862	2,105	1,943	2,186	2,526	2,842
16 km	1,714	1,971	2,228	2,057	2,314	2,674	3,008
17 km	1,809	2,080	2,351	2,170	2,442	2,822	3,174
18 km	1,903	2,189	2,474	2,284	2,569	2,969	3,340
19 km	1,998	2,298	2,597	2,398	2,697	3,117	3,506
20 km	2,093	2,406	2,720	2,511	2,825	3,264	3,672

Table A.2 shows how OpenWeatherMap weather condition groups [27] map to the weather multiplier.

Table A.2: Mapping of OpenWeatherMap condition groups to the weather multiplier

Condition group (ID range)	Main value	M_weather
2xx	Thunderstorm	1.35
3xx	Drizzle	1.20
5xx	Rain	1.20
6xx	Snow (not expected in Sri Lanka)	1.00
7xx	Atmosphere (mist, haze, etc.)	1.00
800	Clear	1.00
80x	Clouds	1.00

Condition group (ID range)	Main value	M_weather
—	Timeout or error	1.00 (fallback)

Table A.3 gives worked calculations that can be used to check the service by hand.

Table A.3: Worked transport fee calculations

Case	Inputs	Calculation	Fee (LKR)
1. Documented API example	D = 5.4 km; heavy traffic; rain; fuel ratio 1.183	(200 + 5.4 × 80 × 1.183) × 1.30 × 1.20 = 711.06 × 1.56 = 1,109.25	1,109
2. Zero distance	D = 0; normal; clear	(200 + 0) × 1.00 × 1.00	200
3. Short trip in thunderstorm	D = 2 km; normal traffic; thunderstorm	(200 + 2 × 80 × 414/350) × 1.00 × 1.35 = 389.257 × 1.35 = 525.497	525
4. Weather service timeout	D = 10 km; moderate traffic; weather unavailable	(200 + 10 × 80 × 414/350) × 1.15 × 1.00 = 1,146.29 × 1.15 = 1,318.23	1,318

Appendix B Academic integrity and AI use disclosure

In line with the module policy, the following use of generative AI and automated tools is disclosed.

Table B.1: Disclosure of AI and automated tool use

Tool	Purpose	Sections or artefacts affected	Verification by the author
Claude (Anthropic), AI assistant	Drafting and structuring thesis text from the author's component specification, results and answers; searching for and formatting literature references; producing diagrams and charts from the author's data	All chapters; figures; reference list	All technical facts, parameter values and results checked against the author's code, outputs and deployment; references checked against their sources

All numerical results reported in Chapter 6 for the repair price models, the latency benchmark and the automated tests were produced by the author's own code and deployment. The transport fee scenario values were calculated directly from the fee formula with the configured parameters. No results were generated by an AI tool. The author remains responsible for the accuracy, originality, citations, code, data, analyses and conclusions of this thesis.